CS 330 SOFTWARE ENGINEERING · CLASS 6 · TUESDAY, OCTOBER 27, 2026

Software Design: Architecture, Patterns & Frameworks

Topic 3 — Software Development Phases (Software Design)

LO2 (Learning Outcome 2) — Apply design patterns and framework activities for software design.

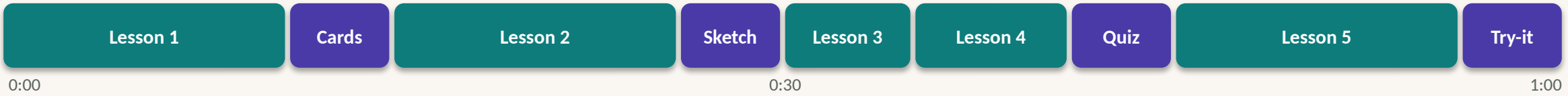
No new Block today: the Studio is coached time on Block 5.

Use of this material: Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0)

Four questions for the hour

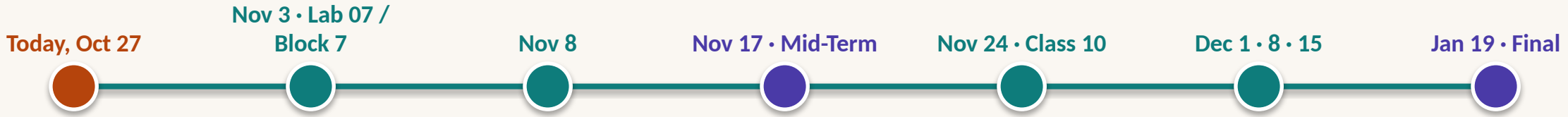
1. What makes a design good — and how do we measure it?
2. Which architecture — and how do we record the choice?
3. Which principles and patterns shape good classes?
4. Which framework — and how do we draw the design for another engineer?

The hour at a glance



Time	Part	Kind	Slides
0:00–0:11	Lesson 1 — Design concepts and design quality	Lesson	4–11
0:11–0:15	Flash-cards — cohesion, coupling, modularity, information hiding	Active break	12
0:15–0:26	Lesson 2 — Architectural design	Lesson	13–20
0:26–0:30	Visualize-it — sketch your product's architecture	Active break	21
0:30–0:35	Lesson 3 — Object-oriented design principles	Lesson	22–27
0:35–0:41	Lesson 4 — Design patterns of the Gang of Four	Lesson	28–35
0:41–0:45	Fun-quiz — name that pattern	Active break	36
0:45–0:56	Lesson 5 — Frameworks and component-level design	Lesson	37–45
0:56–1:00	Try-it — a one-minute sequence diagram, then one quiet minute	Active break	46
1:00–1:10	Break — ten real minutes	—	—
1:10–2:50	Studio — coached work on Block 5 (optional: pattern card, code skeleton)	After the hour	47
2:50–3:00	Wrap-up — exit ticket	After the hour	47
at home	Home Sprint — about 120 minutes	After the hour	48

Where today's ideas come back



Coached Studio: finish Block 5; optional pattern card; the code skeleton Block 7 needs

first code and unit tests on today's structure — each cohesive class is tested alone

recommended first version of Block 7 (block7-draft) for feedback before the Mid-Term

Part A: design concepts, **SOLID** (Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion), styles; Part C: identify a design pattern and sketch its classes (**LO2** (Learning Outcome 2))

ADRs (Architecture Decision Records) as documentation; refactoring and technical debt in maintenance

finishing workshops: review a teammate's file with the design checklist before the tag

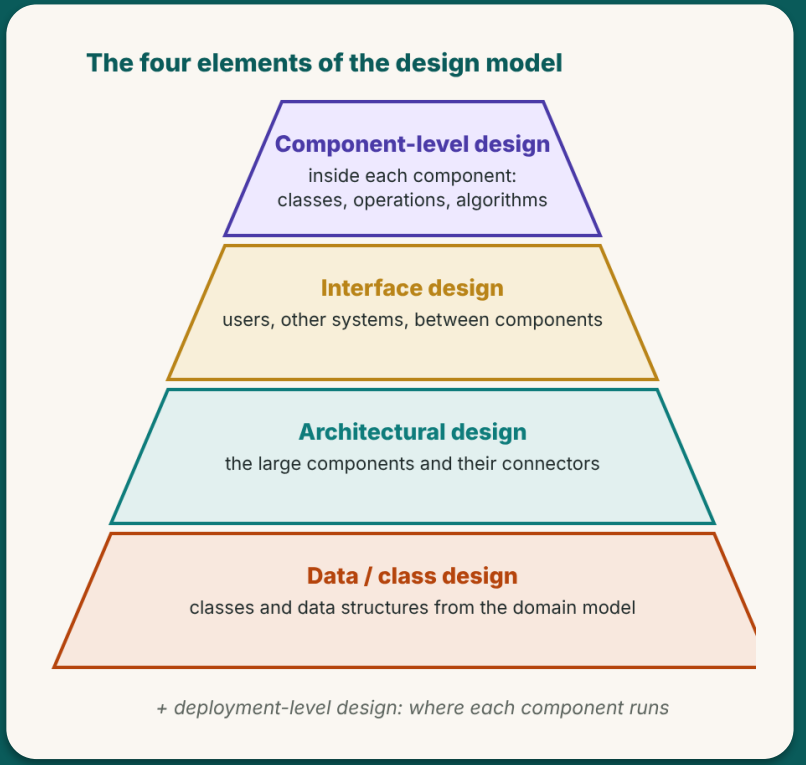
a pattern and its test cases at two levels; a change request through a well-structured design

1 Lesson 1 — Design concepts and design quality

0:00–0:11 · 11 minutes · slides 1–11

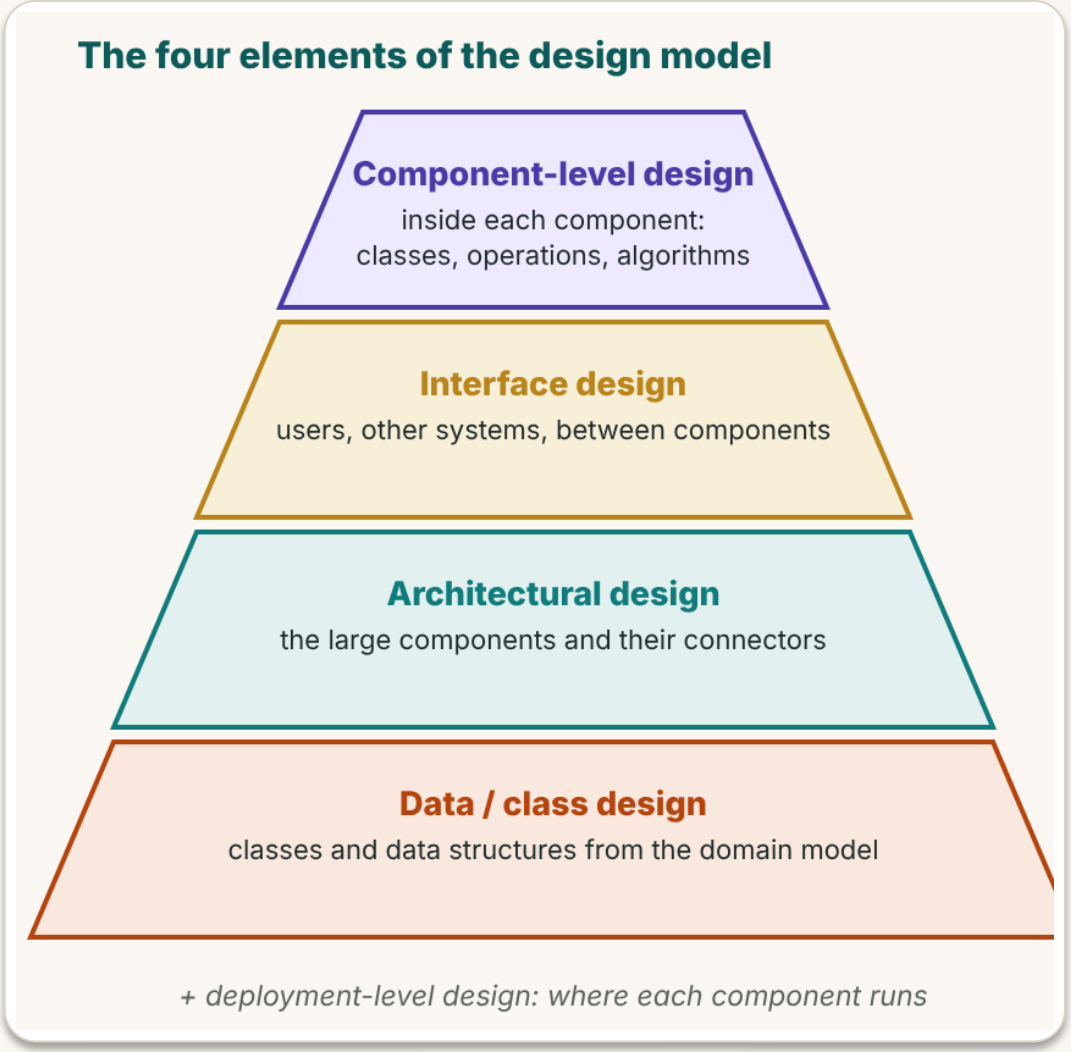
By the end of this lesson you can:

Define software design and the four elements of the design model; explain abstraction, modularity, information hiding and separation of concerns; define cohesion and coupling and measure them on a small design; define refactoring and say why tests make it safe.



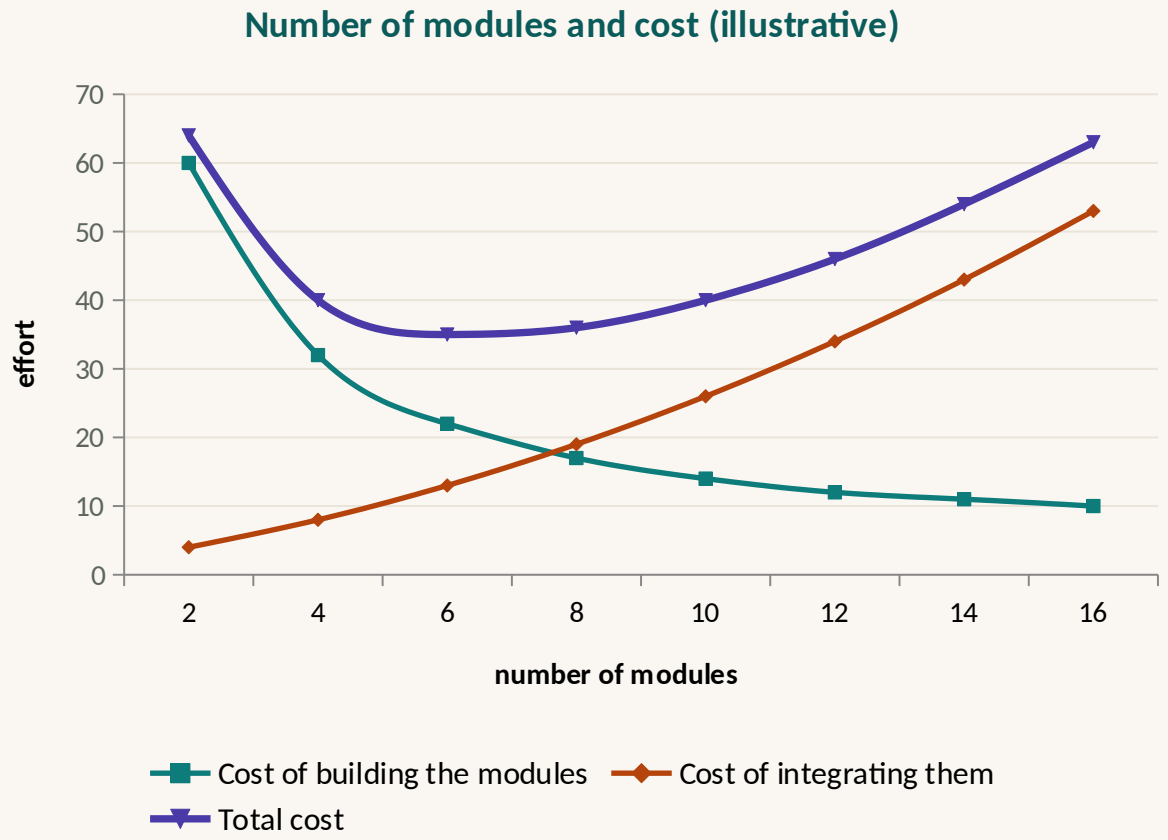
From requirements to design: the design model

- **Software design**: a model of the software — structure, data, interfaces, components — made before the code
- Four elements: data/class design · architectural design · interface design · component-level design (+ deployment)
- Built bottom-up from the requirements model: the domain model feeds the data/class design
- A good design implements every requirement, is readable by coders and testers, and gives a complete picture



Abstraction and modularity

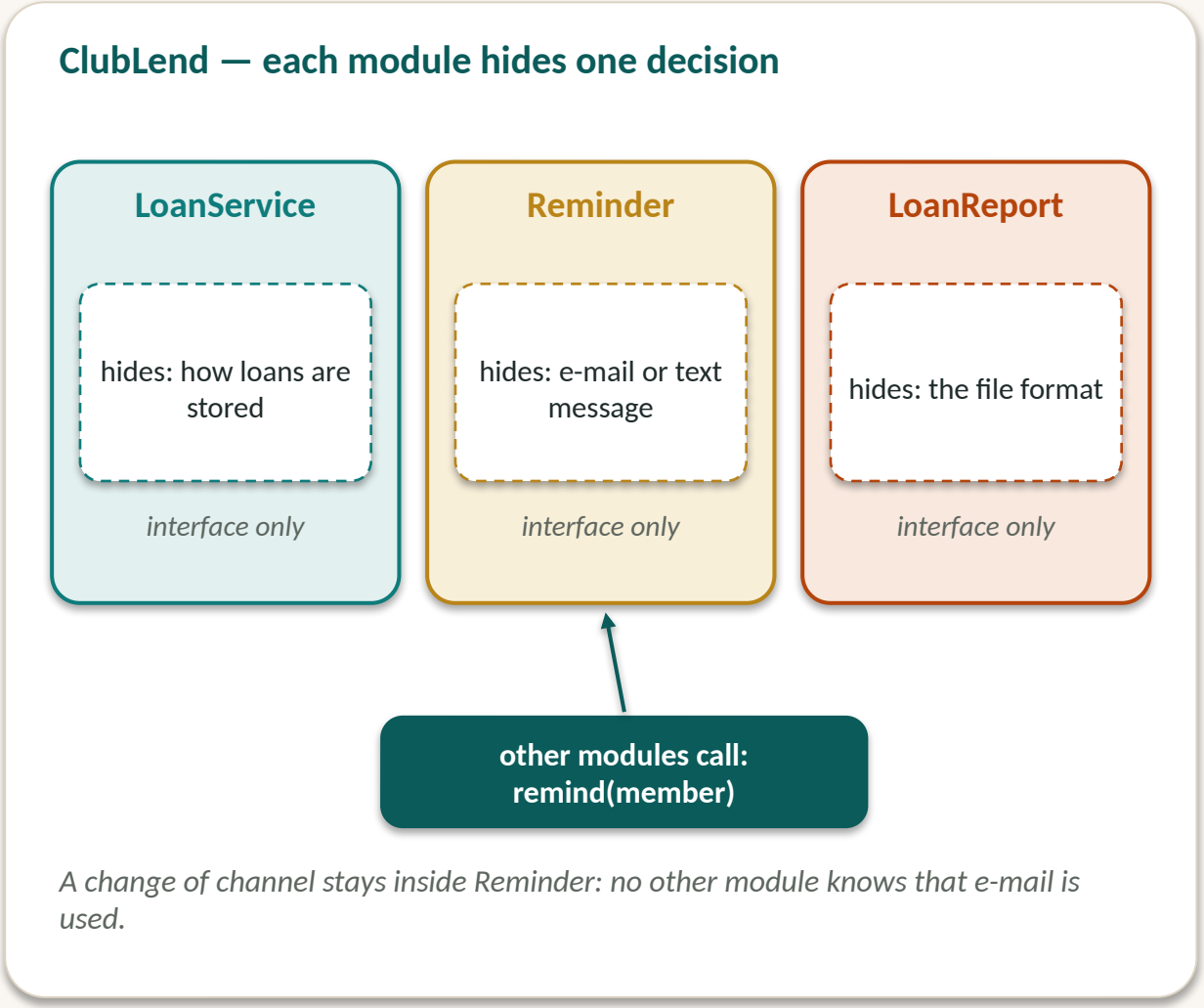
- **Abstraction**: name a concept and ignore the details that do not matter at this level
- Procedural abstraction (*borrow an item*) · data abstraction (*Loan*: member, item, due date)
- **Modularity**: divide the software into separately named, addressable modules
- Too few modules: each is hard to understand · too many: integration cost rises — aim for the middle



The total is lowest in the middle: modules small enough to understand, few enough to integrate.

Information hiding and separation of concerns

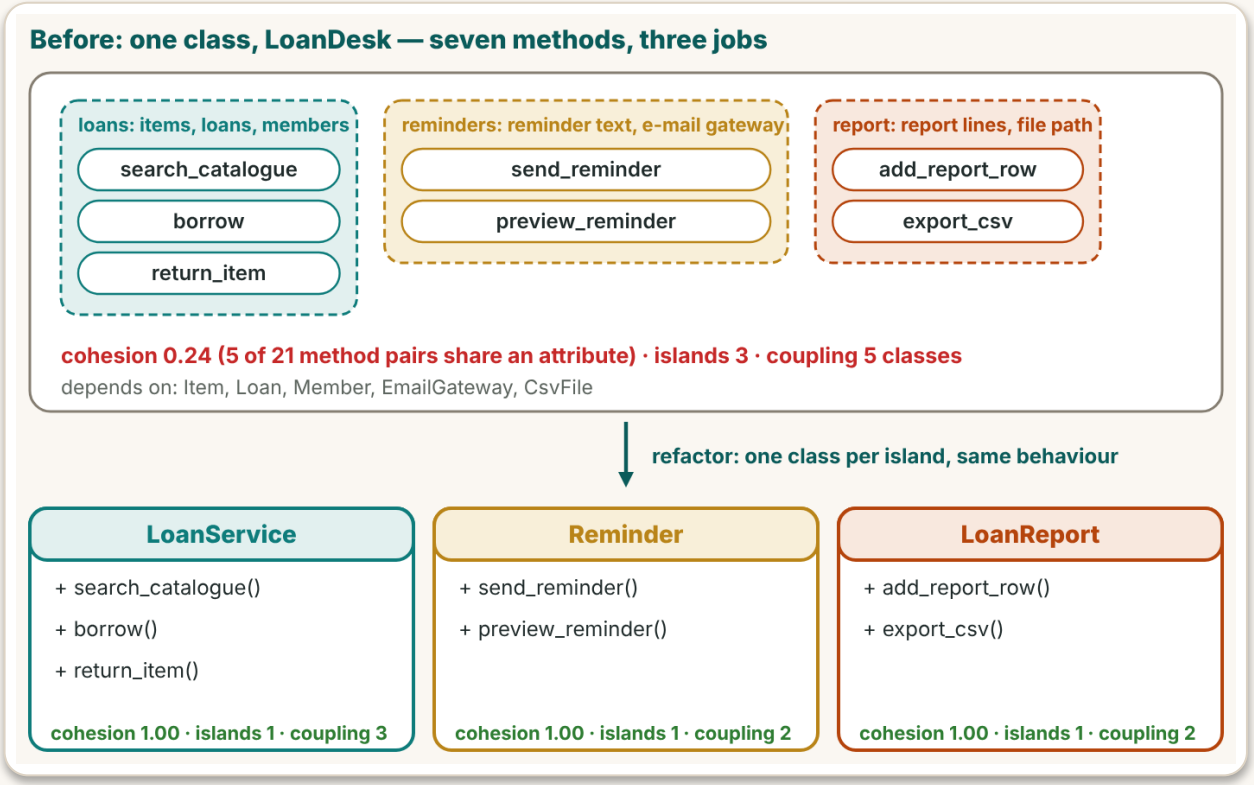
- **Information hiding** (Parnas, 1972): each module hides a design decision that is likely to change
- Other modules use its interface, never its internals
- **Separation of concerns**: divide a problem into parts that can be solved and changed separately
- A **concern** = a feature or behaviour of the requirements (loans, reminders, reports)



A change of channel stays inside Reminder: no other module knows that e-mail is used.

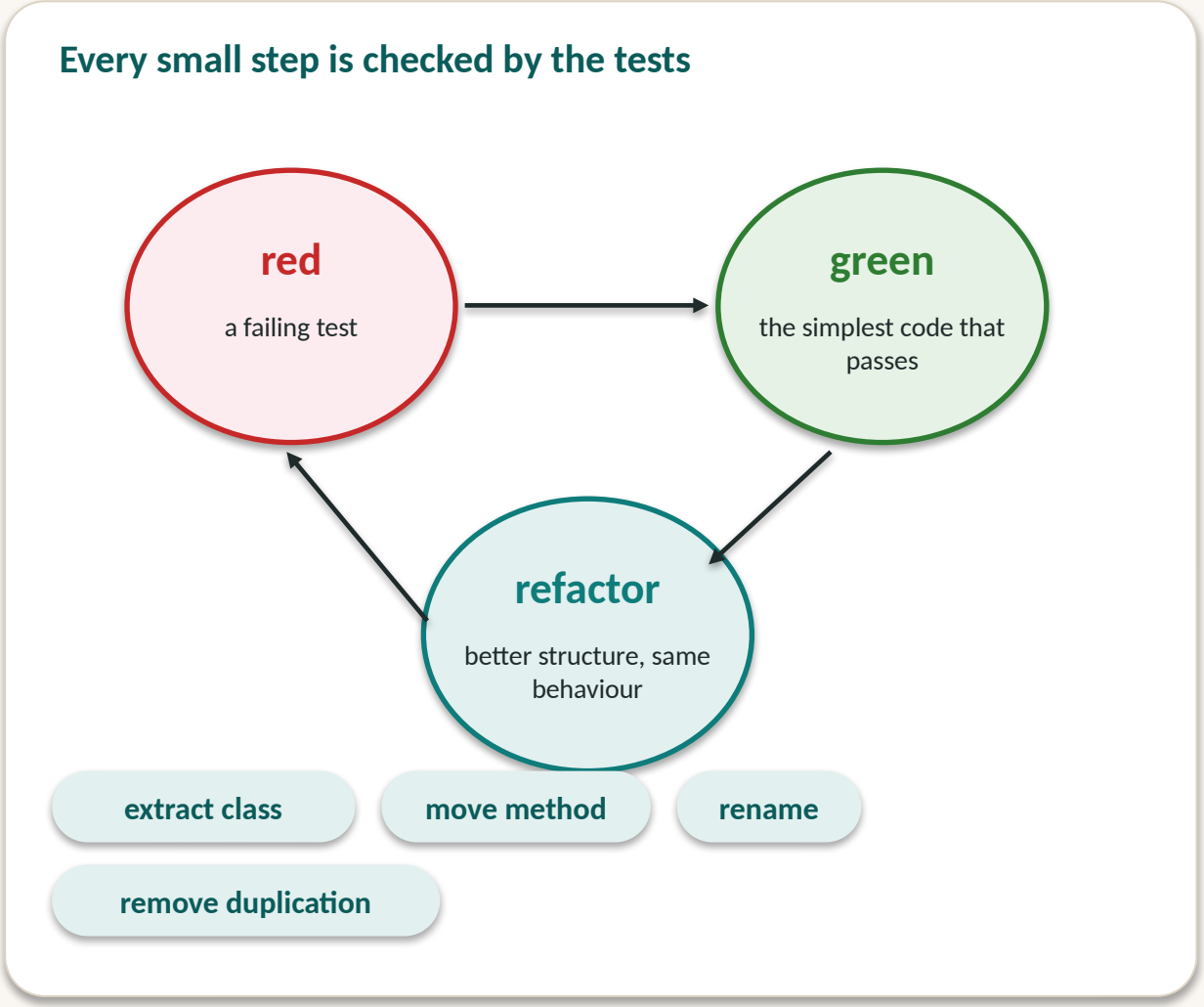
Cohesion and coupling

- **Functional independence**: each module does one job and interacts little with the others
- **Cohesion**: how strongly the elements inside one module belong together → aim HIGH
- **Coupling**: how strongly one module depends on others → aim LOW
- Measurable: **TCC** (Tight Class Cohesion) = linked method pairs ÷ all method pairs; coupling count = classes depended on



Refactoring

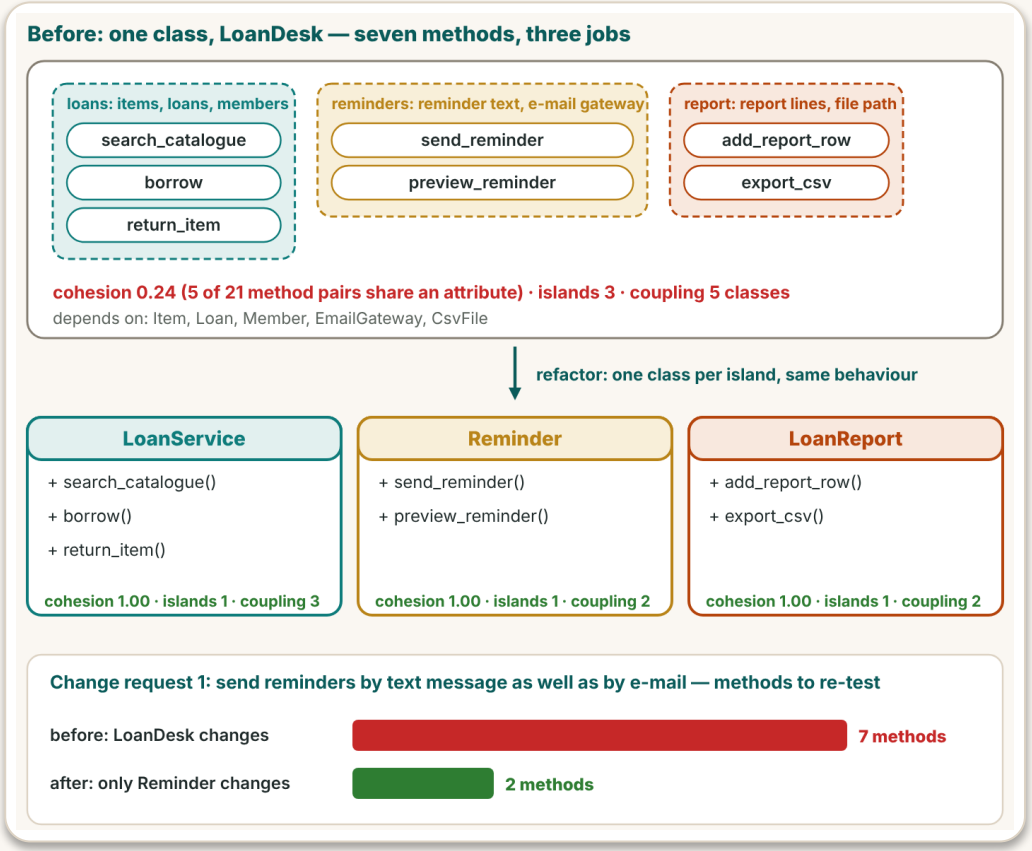
- **Refactoring**: change the internal structure without changing the observable behaviour
- Purpose: easier to understand, cheaper to change
- Small steps, each checked by the tests that already exist (Class 4: red → green → refactor)
- Typical moves: extract a class, move a method, rename, remove duplication



Example — ClubLend (Team Owl, fictional)

one class, three jobs — measured, split, measured again

Task. Team Owl (fictional) builds ClubLend, the lending app of a student club that lends board games and books.



0.24 → 1.00

TCC (Tight Class Cohesion) of the largest class

5 → 3

largest coupling count

7 → 2

methods to re-test for CR-1 (Change Request 1)

Lesson 1 in the module — watch, see it applied, run it, check yourself

Videos — each under 15 minutes

3 checked videos; best match: “Coupling and Cohesion” (Udacity)

Real-life example

Team Owl · ClubLend (fictional): task, relation to the lecture, deliverables that come back in later classes

Program 1 — run it step by step

demo1_clublend_cohesion_coupling.py: Start, Pause, Resume, Step, Restart; every executed step, variable and output line shown; a live visualisation

Figure 1 and the tables

Download the figure as an SVG (Scalable Vector Graphics) or PNG (Portable Network Graphics) file, and the tables as CSV (Comma-Separated Values) or the Class 6 workbook

Five flash-cards

Click to turn; shuffle; the same cards return in the Study Hub

Formative quiz — 4 + 2 questions

Four single-answer and two multiple-answer questions, hints, the reason for every option — formative, never graded

Active break — Flash-cards: Four cards, two minutes each way

- In pairs. Partner A reads the front of four cards from the screen: **cohesion**, **coupling**, **modularity**, **information hiding**.
- Partner B answers each one in one sentence, without notes; A checks against the back.
- Swap roles for the next two minutes with the same four cards in a new order.
- Last 20 seconds: each pair names one class of its own domain model that would have low cohesion if nobody watched it.



0:11–0:15

four minutes · in pairs or alone

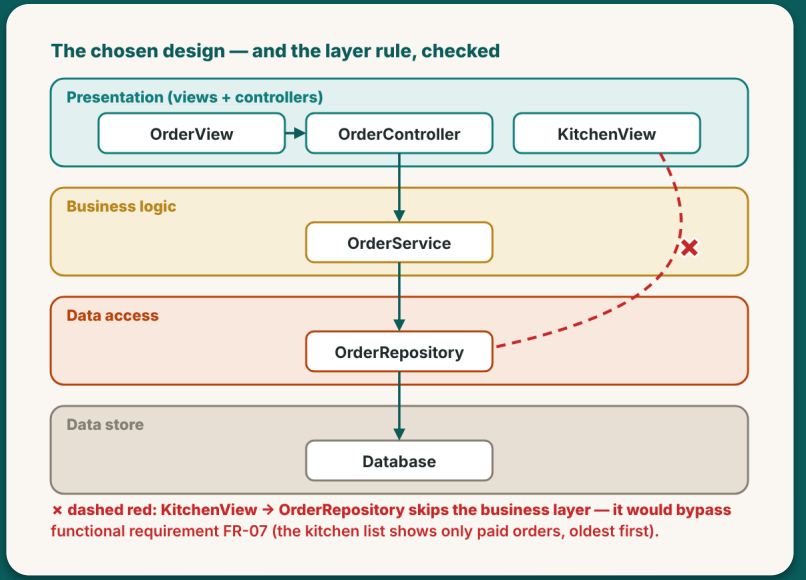
cohesion	coupling
modularity	information hiding

2 Lesson 2 — Architectural design

0:15–0:26 · 11 minutes · slides 13–20

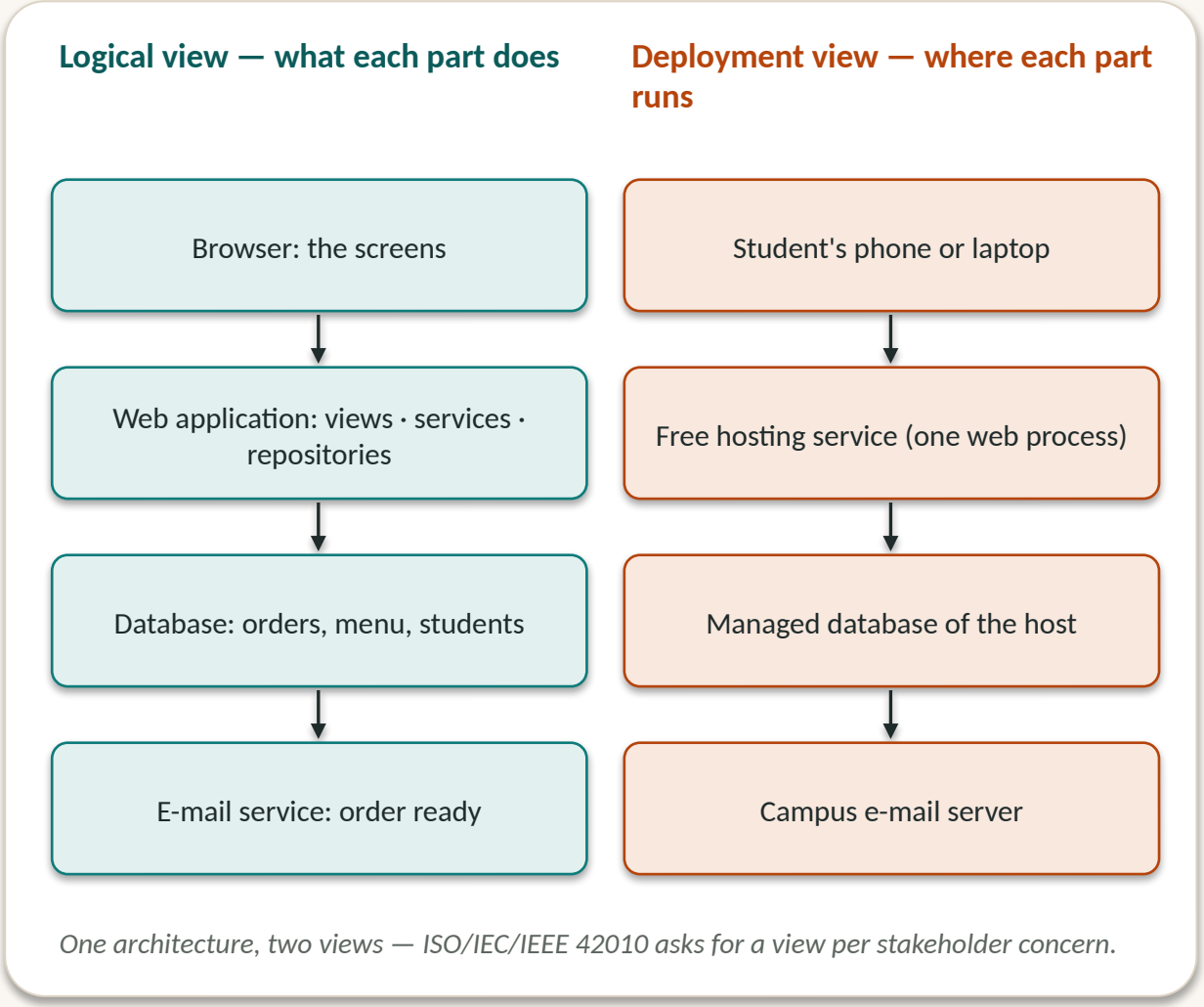
By the end of this lesson you can:

Define software architecture and an architectural style; describe six styles with their strengths and costs; choose a style from weighted decision drivers; record the decision in an **ADR** (Architecture Decision Record); check a design against the rule of the layered style.



What software architecture is

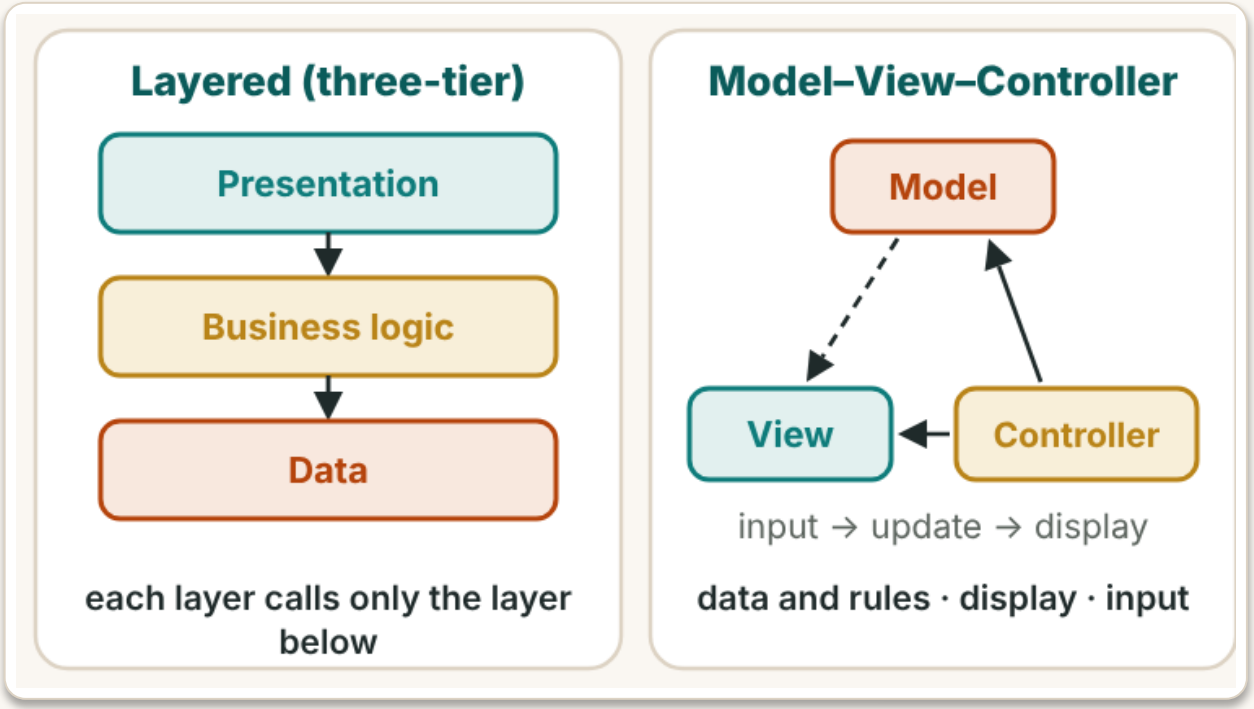
- **Software architecture**: the structure of the system — its elements, their visible properties, their relationships
- Decided early, expensive to change late: it fixes performance, modifiability, testability
- One architecture, several **views** (logical, deployment…) — **ISO/IEC/IEEE 42010** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 42010)
- Architecture = the large boxes; component-level design = the inside of each box



One architecture, two views — ISO/IEC/IEEE 42010 asks for a view per stakeholder concern.

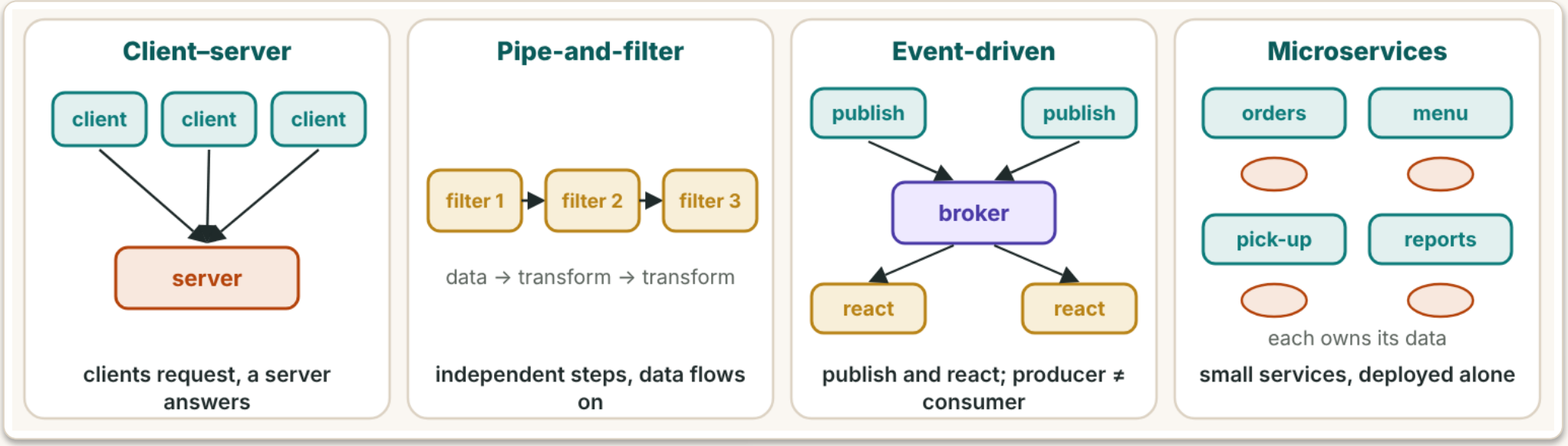
Architectural styles (1/2): layered and MVC (Model–View–Controller)

- **Architectural style** = components + connectors + constraints on how they combine
- **Layered / 3-tier**: presentation → business logic → data; each layer uses only the layer below
- **MVC (Model–View–Controller)**: Model (data, rules) · View (display) · Controller (input) — Reenskaug, Xerox **PARC** (Palo Alto Research Center), 1978–79
- Web frameworks adapt **MVC** (Model–View–Controller) (Django calls it **MTV** (Model–Template–View))



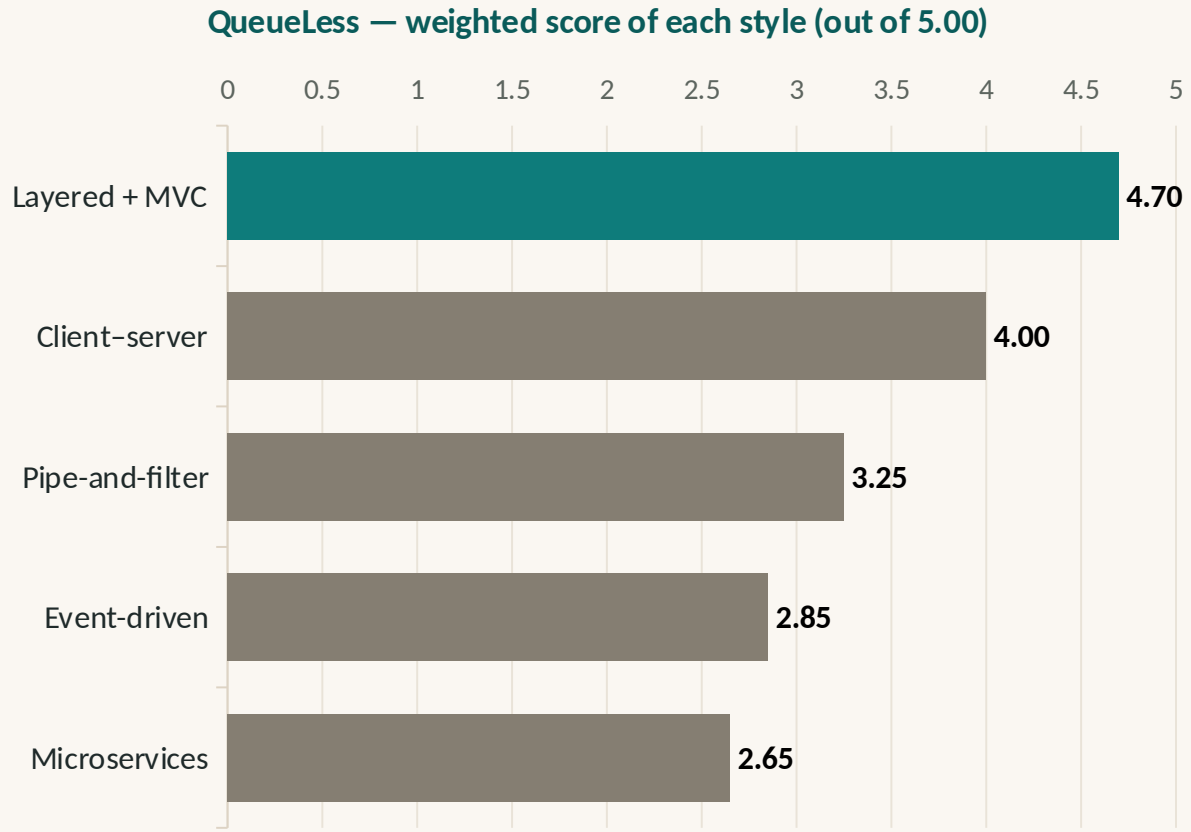
Architectural styles (2/2): client–server, pipe-and-filter, event-driven, microservices

- **Client–server**: clients send requests, a server answers (the web itself)
- **Pipe-and-filter**: data flows through independent processing steps (compilers, data pipelines)
- **Event-driven**: components publish and consume events; producers do not know consumers
- **Microservices**: small services, each deployed alone, around one business capability — powerful, and costly to operate



Choosing a style: drivers and trade-offs

- **Decision drivers**: the requirements and constraints that matter most (Block 5 **NFRs** (Non-Functional Requirements), team, hosting)
- Every style trades one quality for another: no style is best in general
- Weighted matrix: weight per driver (sum 100) × score 1–5 → total ÷ 100
- The matrix makes the judgement explicit and repeatable; the scores stay a judgement



Weights: fit to request–response 25 · team skills 25 · free hosting 20 · testability 15 · noon-rush load 15. MVC = Model–View–Controller.

Recording the decision: the ADR (Architecture Decision Record)

- **ADR (Architecture Decision Record)** (Nygard, 2011): one short file per decision, numbered, kept in the repository
- Title · status · context · decision · consequences (both the gains and the costs)
- Accepted records are not edited: a new decision supersedes an old one
- You wrote one in Block 3 (process); Class 10 makes **ADRs** (Architecture Decision Records) part of the documentation

ADR-001 · architectural style of QueueLess — accepted

Context	Three or four students, one semester, free hosting, about 300 orders between 11:30 and 12:00.
Options	Layered + MVC (Model–View–Controller), client–server, pipe-and-filter, event-driven, microservices.
Decision	Layered + MVC: 4.70 against 4.00 for client–server in the weighted matrix.
Consequences	One deployable unit and one database; every feature scales together; revisit if a second campus joins.

One short file per decision, kept with the code (Nygard, 2011).

Example — QueueLess (Team Falcon, fictional)

five styles, five drivers, one decision record — and one layer violation

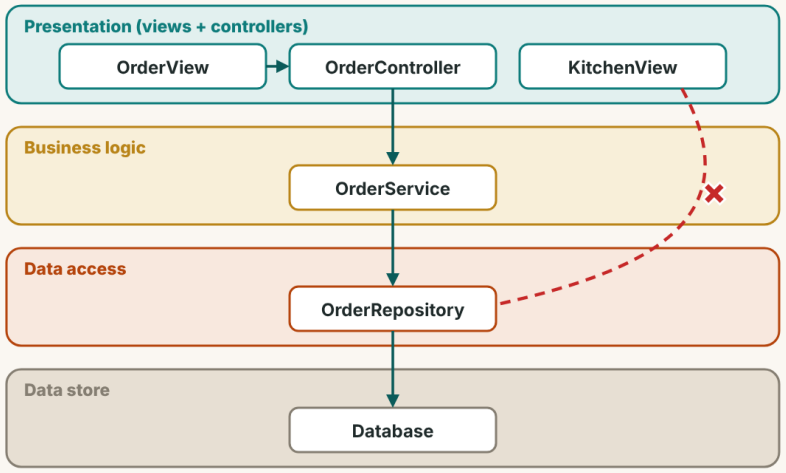
Task. Team Falcon (fictional) builds QueueLess, the cafeteria pre-order app: a student orders before the noon rush and picks up the order when the number is called; the kitchen sees the queue.

QueueLess — weighted decision matrix (weights add up to 100; scores 1–5)

style	fit weight 25	skills weight 25	hosting weight 20	testing weight 15	load weight 15	weighted score
Layered + Model–View–Controller	5	5	5	4	4	**4.70**
Client–server	5	3	4	4	4	4.00
Pipe-and-filter	1	4	4	5	3	3.25
Event-driven	3	2	2	3	5	2.85
Microservices	4	1	1	3	5	2.65

Score = sum of (weight × score) ÷ 100. The highest total becomes the decision record ADR-001 of the team.

The chosen design — and the layer rule, checked



× dashed red: KitchenView → OrderRepository skips the business layer — it would bypass functional requirement FR-07 (the kitchen list shows only paid orders, oldest first).

Relation to the lecture. It applies Lesson 2: architectural styles, decision drivers and their weights, the **ADR** (Architecture Decision Record), and the constraint that defines the layered style.

4.70

Layered + MVC (Model–View–Controller) — the chosen style

4.00

client–server, the runner-up

1

layer violation found before any code

Lesson 2 in the module — watch, see it applied, run it, check yourself

Videos — each under 15 minutes

4 checked videos; best match: “Top 5 Most Used Architecture Patterns” (ByteByteGo)

Real-life example

Team Falcon · QueueLess (fictional): task, relation to the lecture, deliverables that come back in later classes

Program 2 — run it step by step

demo2_queueless_architecture_decision.py: Start, Pause, Resume, Step, Restart; every executed step, variable and output line shown; a live visualisation

Figure 2 and the tables

Download the figure as an SVG (Scalable Vector Graphics) or PNG (Portable Network Graphics) file, and the tables as CSV (Comma-Separated Values) or the Class 6 workbook

Five flash-cards

Click to turn; shuffle; the same cards return in the Study Hub

Formative quiz — 4 + 2 questions

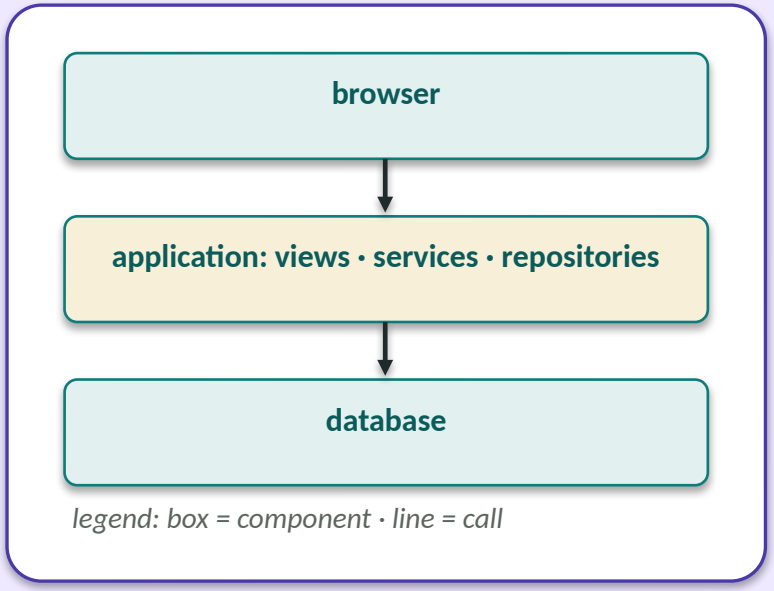
Four single-answer and two multiple-answer questions, hints, the reason for every option — formative, never graded

Active break — Visualize-it: Sketch your product's architecture

- Two minutes, alone, on paper: draw your product as boxes and lines — the browser, the application with its layers, the data store, any external service (e-mail, campus login).
- Add a three-line legend: what a box means, what a line means, what a dashed line means.
- One minute with a neighbour from another team: which style is it? Does any line skip a layer?
- Last minute: write the name of the style and one sentence of reason — the first line of a future **ADR** (Architecture Decision Record).

👁 **0:26–0:30**

four minutes · in pairs or alone

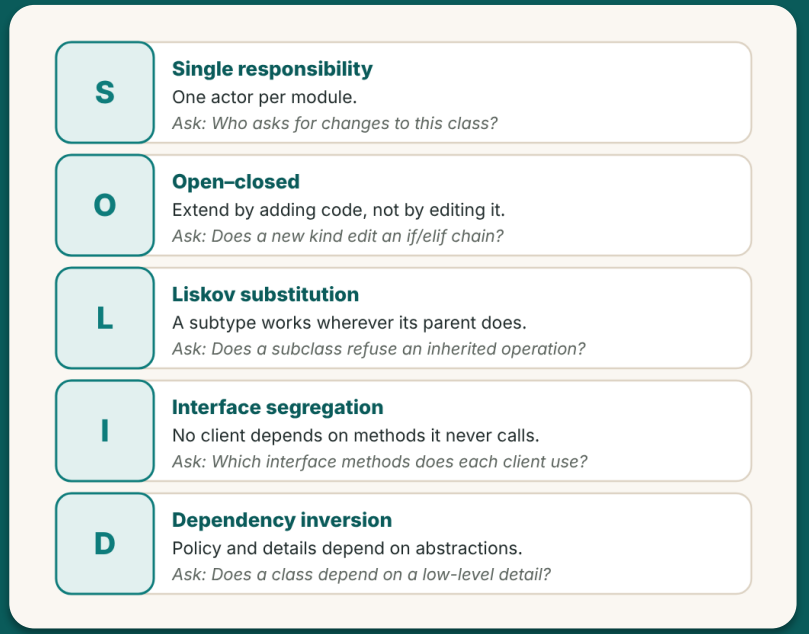


3 Lesson 3 — Object-oriented design principles

0:30–0:35 · 5 minutes · slides 22–27

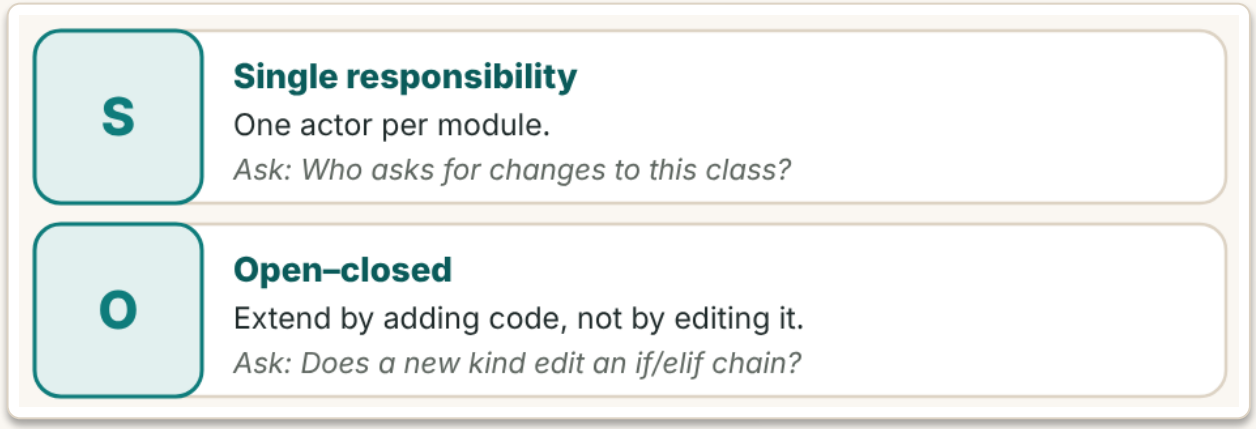
By the end of this lesson you can:

State the five **SOLID** (Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion) principles and **DRY** (Don't Repeat Yourself), **KISS** (Keep It Simple, Stupid) and **YAGNI** (You Aren't Gonna Need It) in one sentence each; recognise a violation of each in a small design; explain how each principle lowers the cost of a change.



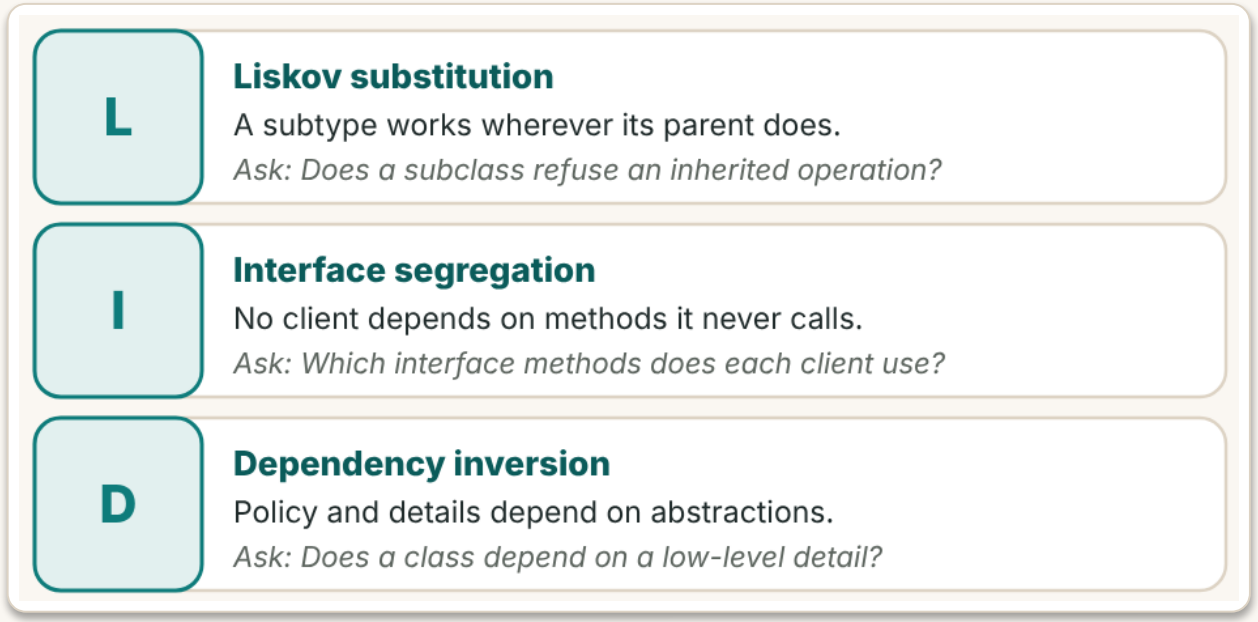
SRP (Single Responsibility Principle) and OCP (Open–Closed Principle)

- **SOLID** (Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion): five principles collected by Robert C. Martin; the acronym by Michael Feathers
- **SRP (Single Responsibility Principle)**: a module should be responsible to one, and only one, actor — one reason to change
- **OCP (Open–Closed Principle)**: open for extension, closed for modification (Bertrand Meyer, 1988)
- Sign of a violation: an if/elif chain on a kind that grows with every new kind



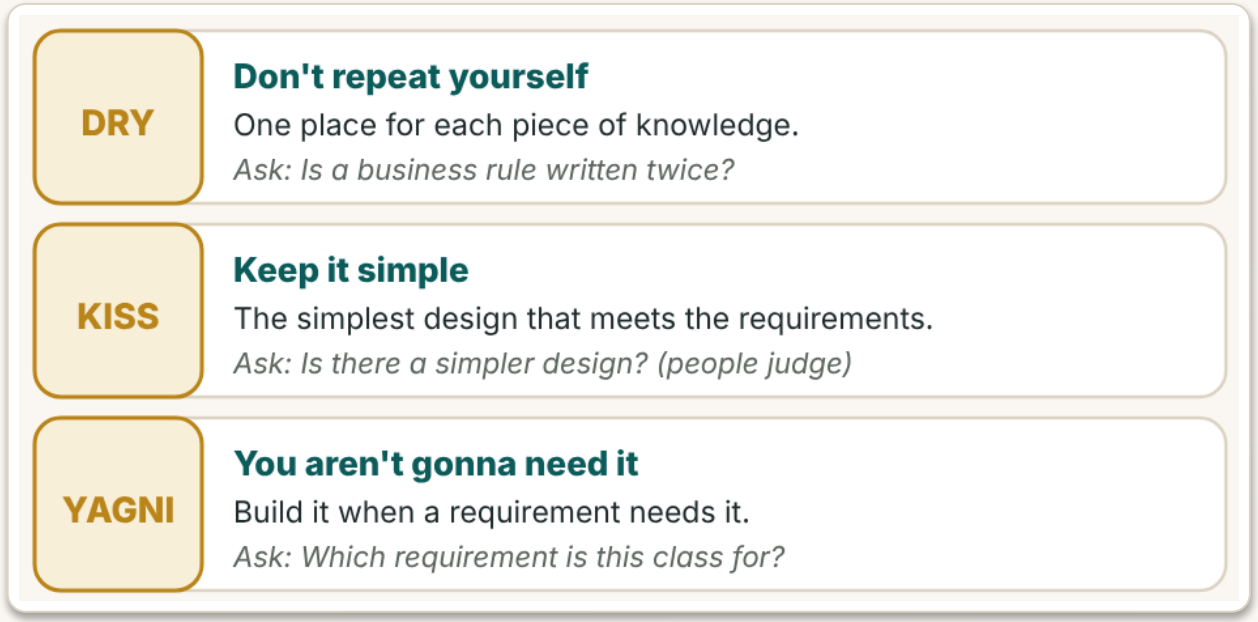
LSP (Liskov Substitution Principle), ISP (Interface Segregation Principle) and DIP (Dependency Inversion Principle)

- **LSP (Liskov Substitution Principle)**: a subtype must be usable wherever its parent type is expected (Liskov and Wing, 1994)
- **ISP (Interface Segregation Principle)**: no client should be forced to depend on methods it does not use
- **DIP (Dependency Inversion Principle)**: high-level policy depends on abstractions; details implement the abstractions
- All three keep a change in one class from forcing changes in others



DRY (Don't Repeat Yourself), KISS (Keep It Simple, Stupid) and YAGNI (You Aren't Gonna Need It)

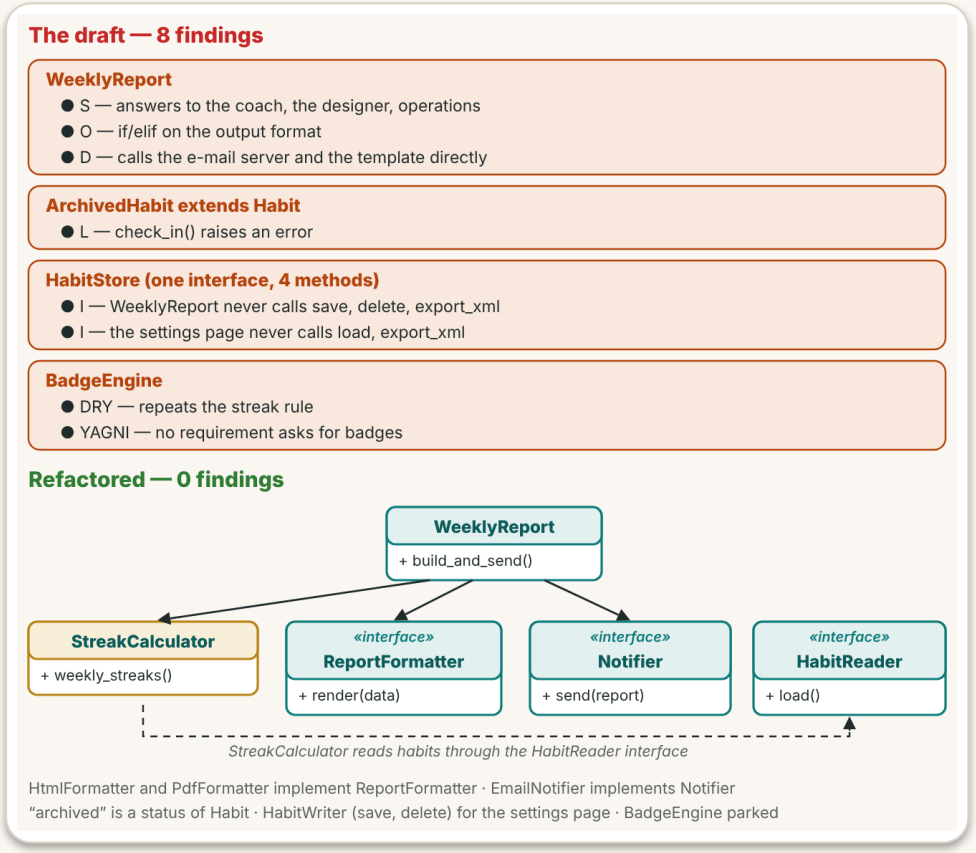
- **DRY (Don't Repeat Yourself)**: every piece of knowledge has one authoritative representation (Hunt and Thomas, 1999)
- **KISS (Keep It Simple, Stupid)**: the simplest design that meets the requirements (you met it in Class 2)
- **YAGNI (You Aren't Gonna Need It)**: build a feature when a requirement needs it, not when you foresee it (Extreme Programming)
- Traceability checks **YAGNI** (You Aren't Gonna Need It): a class with no requirement behind it is speculative work



Example — HabitHub (Team Wren, fictional)

a design review with eight findings — and a refactored design with none

Task. Team Wren (fictional) builds HabitHub: students track daily habits and streaks, and a weekly report shows what held and what slipped.



8

findings in the first draft

0

findings after the refactoring

1

question left to people: KISS (Keep It Simple, Stupid)

Lesson 3 in the module — watch, see it applied, run it, check yourself

Videos — each under 15 minutes

3 checked videos; best match: “SOLID Principles: Do You Really Understand Them?” (Alex Hyett)

Real-life example

Team Wren · HabitHub (fictional): task, relation to the lecture, deliverables that come back in later classes

Program 3 — run it step by step

demo3_habithub_solid_check.py: Start, Pause, Resume, Step, Restart; every executed step, variable and output line shown; a live visualisation

Figure 3 and the tables

Download the figure as an SVG (Scalable Vector Graphics) or PNG (Portable Network Graphics) file, and the tables as CSV (Comma-Separated Values) or the Class 6 workbook

Five flash-cards

Click to turn; shuffle; the same cards return in the Study Hub

Formative quiz — 4 + 2 questions

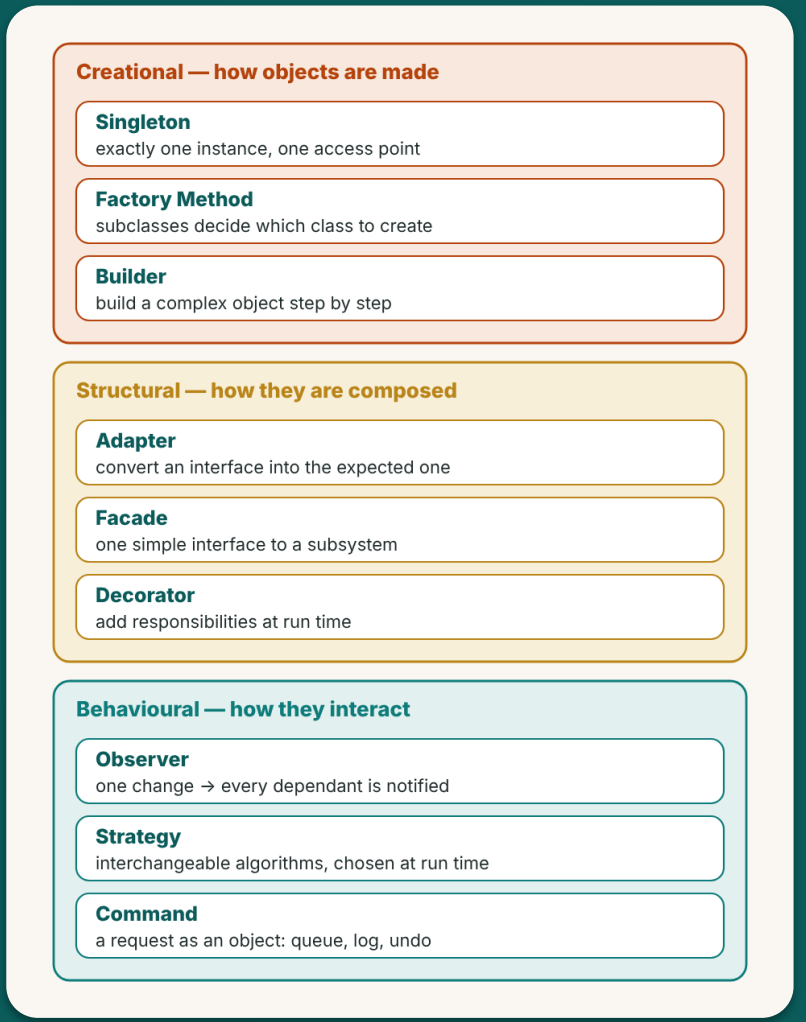
Four single-answer and two multiple-answer questions, hints, the reason for every option — formative, never graded

4 Lesson 4 — Design patterns

0:35–0:41 · 6 minutes · slides 28–35

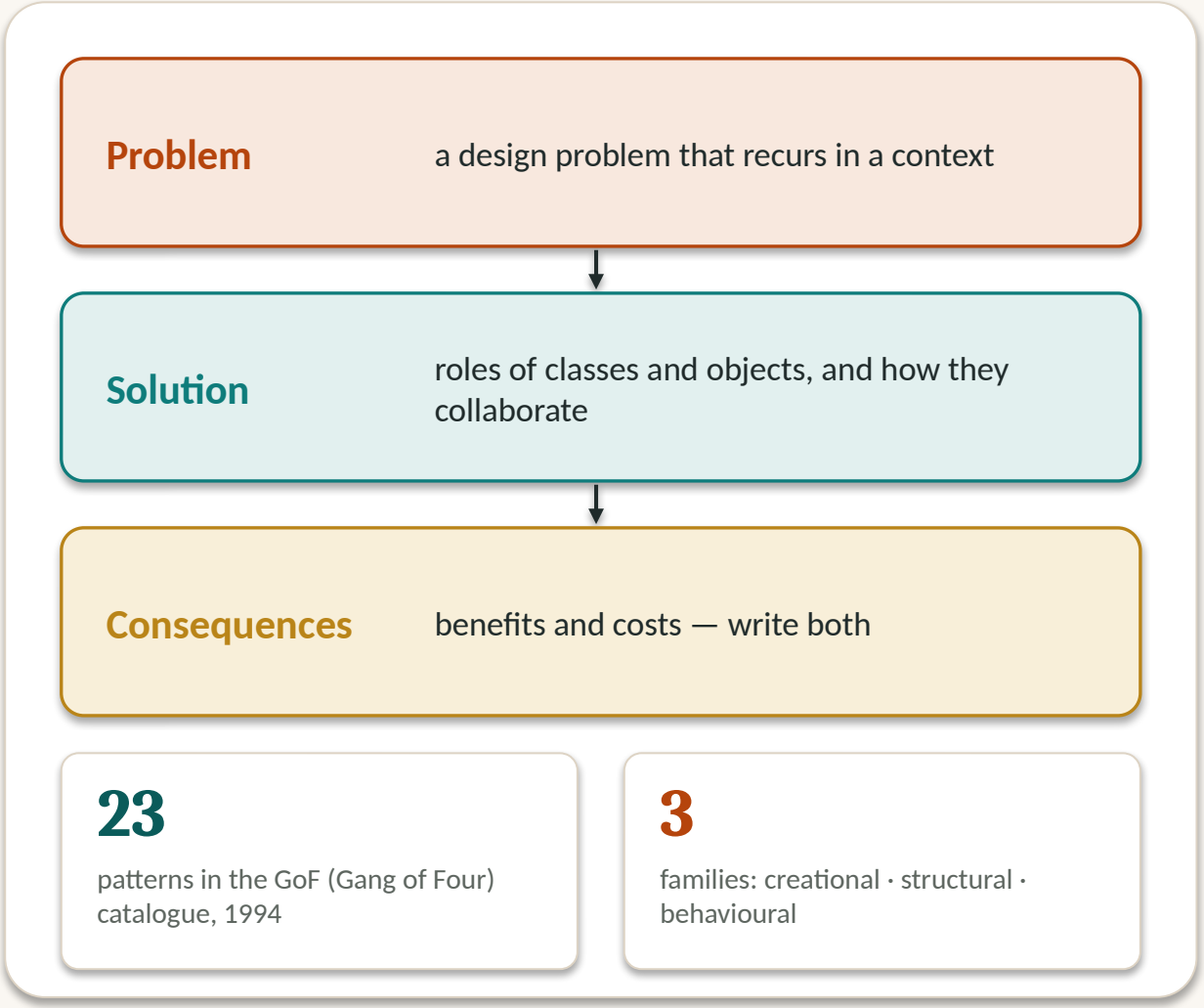
By the end of this lesson you can:

Define a design pattern by its problem, solution and consequences; state the intent of nine **GoF** (Gang of Four — Gamma, Helm, Johnson and Vlissides) patterns and recognise them in short scenarios; describe the structure of Observer and Strategy; choose a pattern from a problem and record it on a pattern card.



What a design pattern is

- **Design pattern**: a named, proven solution to a recurring design problem in a context
- Three parts: the **problem** · the **solution** (classes, roles, collaboration) · the **consequences**
- Christopher Alexander (architecture, 1977) → Gamma, Helm, Johnson, Vlissides — the **GoF** (Gang of Four — Gamma, Helm, Johnson and Vlissides) (1994): 23 patterns
- Creational (how objects are made) · structural (how they are composed) · behavioural (how they interact)



Nine patterns you will meet (1/2): creational and structural

Family	Pattern	Intent, in one line	Where in a campus app
Creational	**Singleton**	Guarantee that a class has exactly one instance and give one global point of access to it.	The configuration of the app, read once
Creational	**Factory Method**	Declare an operation that creates an object, and let subclasses decide which concrete class to create.	A report of the right type for each request
Creational	**Builder**	Separate the step-by-step construction of a complex object from its representation, so one process can build different results.	The weekly report, assembled section by section
Structural	**Adapter**	Convert the interface of an existing class into the interface the client expects.	Wrap the campus login library behind your own interface
Structural	**Facade**	Provide one simple interface to a set of interfaces in a subsystem.	One NotificationFacade in front of e-mail, text and push services
Structural	**Decorator**	Attach extra responsibilities to an object at run time, as a flexible alternative to subclassing.	Add logging or caching around a repository

Singleton creates global state: it makes classes harder to test alone — use it for what really exists once, such as the configuration, and nowhere else.

Nine patterns you will meet (2/2): behavioural

Family	Pattern	Intent, in one line	Where in a campus app
Behavioural	**Observer**	Define a one-to-many dependency so that when one object changes state, all its dependants are informed and updated automatically.	A report changes status → the reporter, the board and the statistics react
Behavioural	**Strategy**	Define a family of algorithms, put each one in its own class, and make them interchangeable for the code that uses them.	Inform a reporter by e-mail or on a tracking page
Behavioural	**Command**	Represent a request as an object, so that requests can be stored, queued, logged or undone.	An “undo the last check-in” button in HabitHub

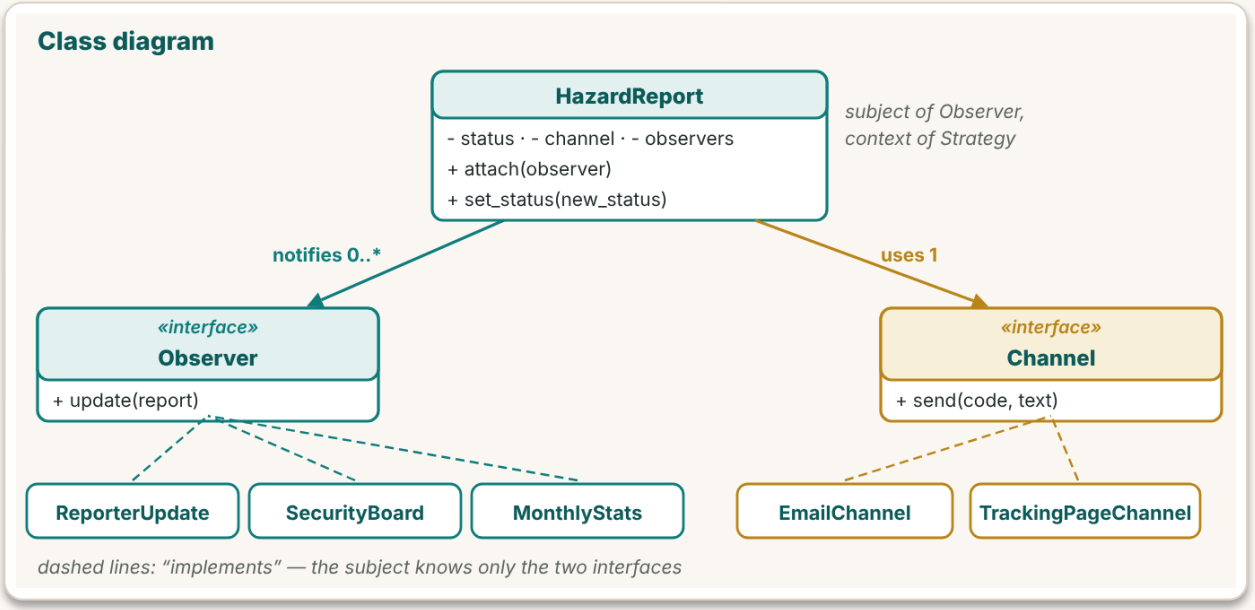
a pattern = problem + solution + consequences

never decoration

Behavioural patterns decide **how objects talk and share work**. Observer and Strategy come back on the next slide, in SafeCampus; Command returns with the undo button of HabitHub.

Observer and Strategy, in detail

- Observer: **subject** keeps a list of **observers**; on change it calls update() on each
- The subject knows only the observer interface → low coupling; add observers without editing it
- Strategy: the **context** holds one **strategy** object; all strategies share one method
- No if/elif on the kind: the object chosen does the work → **OCP** (Open–Closed Principle) in practice



Choosing a pattern — and the pattern card

- Start from a problem you have, never from a pattern you like
- Read the intent and the consequences; compare two candidates
- Pattern card: **Problem → Pattern → Where in our code → Consequences**
- Two patterns that solve real problems beat six that solve none

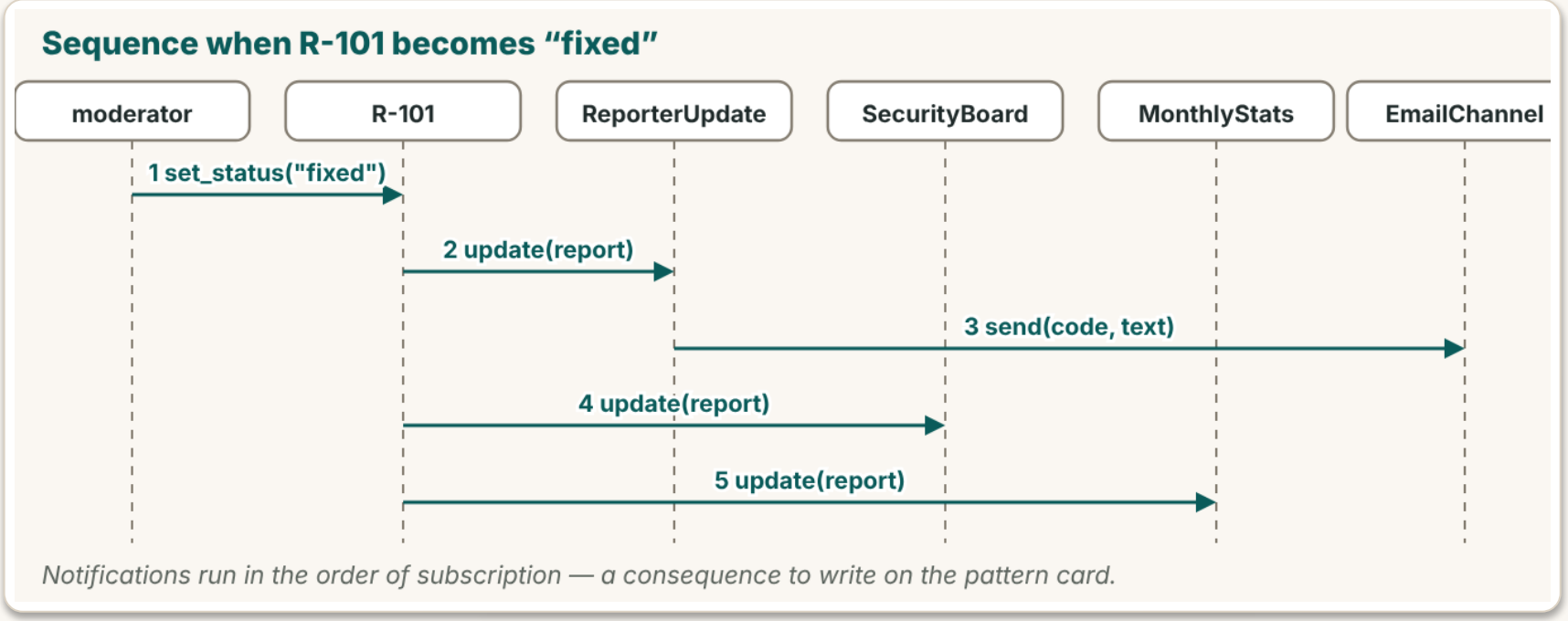
Pattern card — SafeCampus, Team Heron (fictional)

Problem	When a report changes status, several parts must react; the report class must not know them.
Pattern	Observer — HazardReport is the subject; the reactions are observers.
Where in our code	HazardReport · ReporterUpdate · SecurityBoard · MonthlyStats
Consequences	+ new reactions without editing HazardReport · − the order of notifications is the order of subscription; test it.

Example — SafeCampus (Team Heron, fictional)

one status change, three reactions, two channels — Observer and Strategy

Task. Team Heron (fictional) builds SafeCampus: a student or a staff member reports a hazard — a broken stair light, a wet floor — under their name or anonymously, and follows its status until it is fixed.



3

reactions to one status change (observers)

2

channels to tell the reporter (strategies)

0

changes to HazardReport when MonthlyStats is added

Relation to the lecture. It applies Lesson 4: patterns chosen from problems, the roles of Observer and Strategy, and the consequences that go on the pattern card.

Lesson 4 in the module — watch, see it applied, run it, check yourself

Videos — each under 15 minutes

3 checked videos; best match: “10 Design Patterns Explained in 10 Minutes” (Fireship)

Real-life example

Team Heron · SafeCampus (fictional): task, relation to the lecture, deliverables that come back in later classes

Program 4 — run it step by step

demo4_safecampus_observer_strategy.py: Start, Pause, Resume, Step, Restart; every executed step, variable and output line shown; a live visualisation

Figure 4 and the tables

Download the figure as an SVG (Scalable Vector Graphics) or PNG (Portable Network Graphics) file, and the tables as CSV (Comma-Separated Values) or the Class 6 workbook

Five flash-cards

Click to turn; shuffle; the same cards return in the Study Hub

Formative quiz — 4 + 2 questions

Four single-answer and two multiple-answer questions, hints, the reason for every option — formative, never graded

Active break — Fun-quiz: Name that pattern — three scenarios

- Scenario 1: “The whole app must read one configuration object, created once.” Show 1 finger for Singleton, 2 for Factory Method, 3 for Adapter.
- Scenario 2: “When an order is marked ready, the student's screen and the kitchen statistics must both update — and the order class must not know them.” 1 Observer, 2 Command, 3 Facade.
- Scenario 3: “We must use a third-party campus-login library whose methods do not match the interface our controllers expect.” 1 Decorator, 2 Adapter, 3 Strategy.
- Thirty seconds per scenario: vote, then the answer in one sentence.

🎯 **0:41–0:45**

four minutes · in pairs or alone

Scenario 1 — one configuration object

Scenario 2 — ready order, two screens update

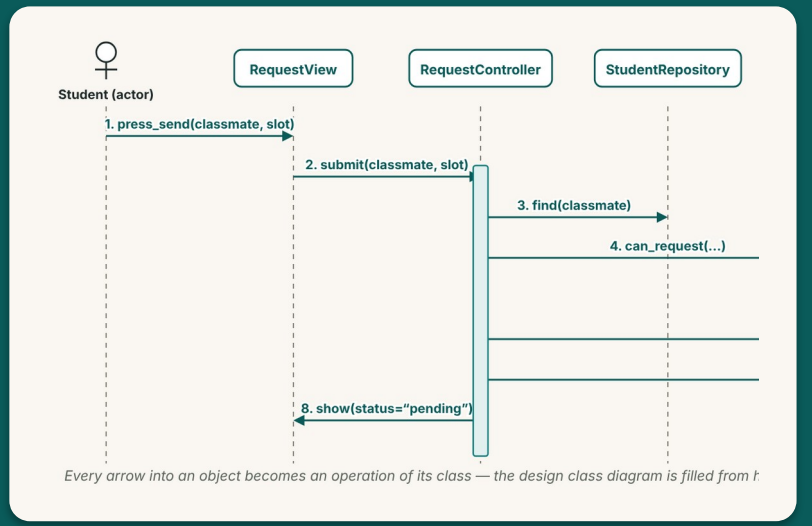
Scenario 3 — a login library that does not fit

5 Lesson 5 — Frameworks and component-level design

0:45–0:56 · 11 minutes · slides 37–45

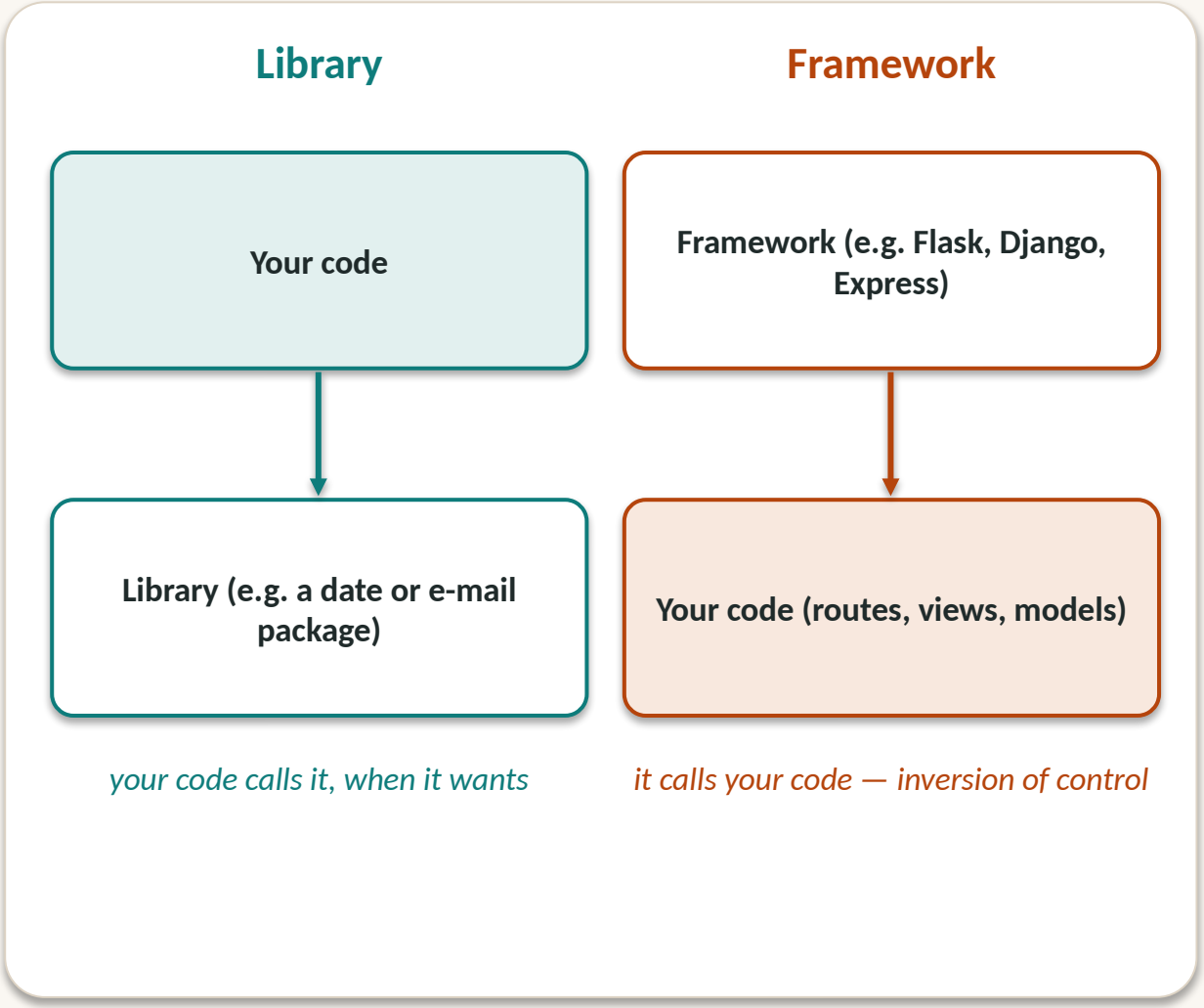
By the end of this lesson you can:

Distinguish a framework from a library; choose a framework with a weighted criteria matrix; derive the operations of a design class diagram from a sequence diagram; read and draw a sequence diagram; state the three golden rules of interface design; compute fan-in and fan-out.



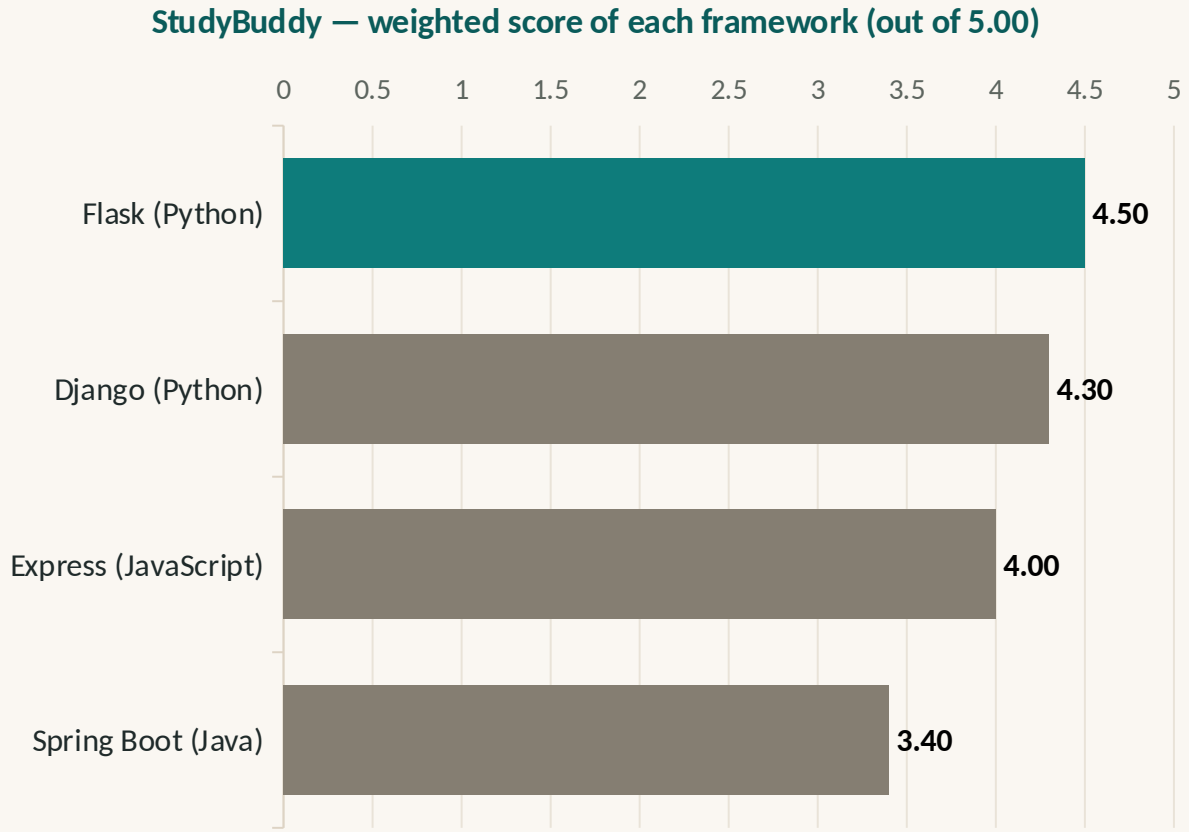
Framework or library?

- **Library**: your code calls it when it needs it
- **Framework**: it calls your code — **inversion of control** — and fixes the structure of the application
- Choosing a framework is an architecture decision: it is hard to change later
- Python: Flask (small), Django (complete) · JavaScript: Express · Java: Spring Boot



Choosing a framework: the criteria matrix

- Criteria: team skills · documentation · testing support · fit for the product · community and support period
- Same method as Lesson 2: weights (sum 100) × scores 1–5 ÷ 100
- Team skills usually weigh most for a four-week build
- Any decision is acceptable if the matrix is honest — write it as an **ADR** (Architecture Decision Record)

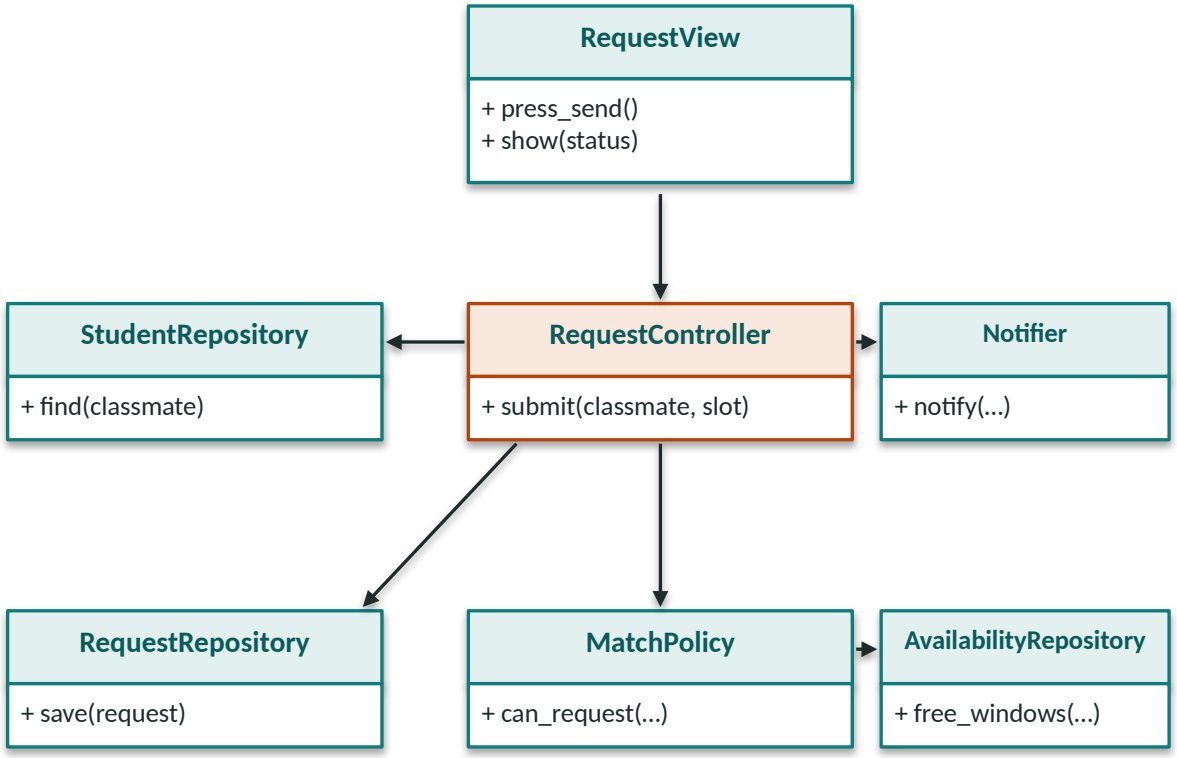


Weights: team skills 30 · documentation 20 · testing support 20 · fit for a small web app 20 · community 10.

Component-level design and the design class diagram

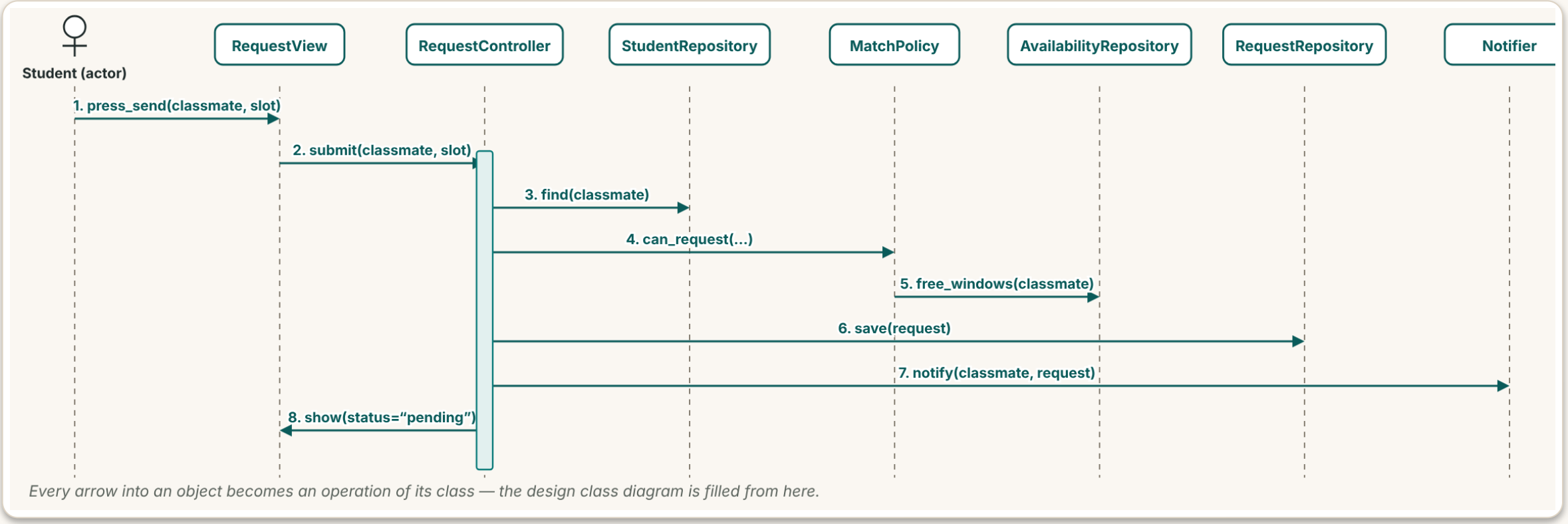
- **Component**: a modular, deployable part with interfaces; designed in detail at the last level
- Domain model (Block 5) → **design class diagram**: + operations, + visibility, + types, + design classes
- Design classes: views/controllers, services, repositories, interfaces — the ones Block 5 banned
- Each operation comes from a message in a sequence diagram

StudyBuddy — design classes, operations taken from the arrows



Arrows: who calls whom. RequestController calls five classes — the largest fan-out of the design.

Sequence diagrams



- Shows the messages exchanged by objects for ONE scenario of ONE use case, in time order
- **Lifelines** (vertical) · **messages** (arrows, numbered) · returns (dashed) · activation bars
- Fragments for conditions and loops: alt, opt, loop
- Read it top to bottom; every arrow into an object = an operation of its class

UI (User Interface) and UX (User Experience) basics: three golden rules

- **UI (User Interface)**: the means by which a person interacts with the product · **UX (User Experience)**: the person's whole experience of it
- Mandel's golden rules: place the user in control · reduce the user's memory load · make the interface consistent
- Accessibility is a requirement, not a decoration: **WCAG** (Web Content Accessibility Guidelines) 2.2 (**W3C** (World Wide Web Consortium))
- Check a screen against the rules before you code it

1 **Place the user in control**

Undo, cancel, no forced modes; the user decides the pace.

2 **Reduce the user's memory load**

Show options instead of asking to remember them; sensible defaults.

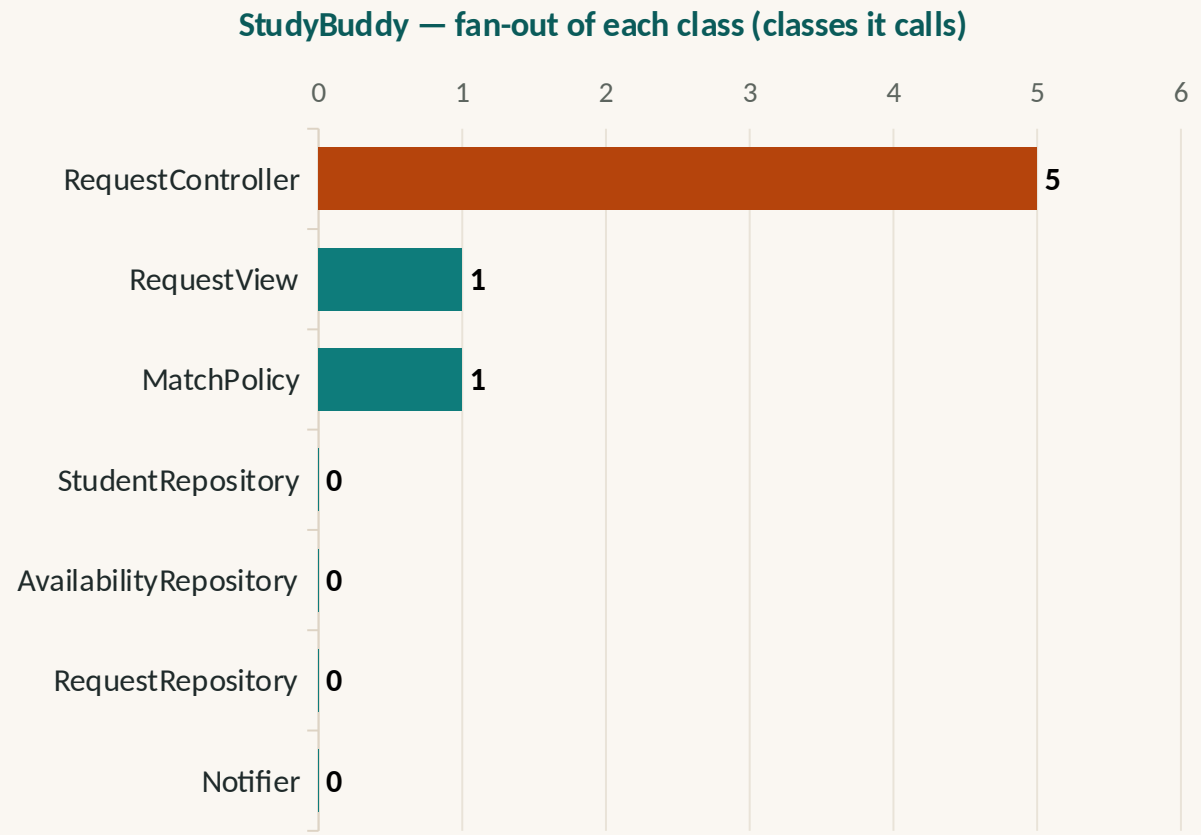
3 **Make the interface consistent**

Same words, same places, same behaviour on every screen.

Check every screen against **WCAG** (Web Content Accessibility Guidelines) 2.2 and the ten usability heuristics of the Nielsen Norman Group.

Numbers corner: fan-in, fan-out and the coupling count

- **Fan-in** of a class = number of different classes that call it (Henry and Kafura, 1981)
- **Fan-out** = number of different classes it calls · **coupling count** = classes it depends on
- High fan-in: widely reused → change with care · high fan-out: depends on much → hard to test alone
- Practical rule: fan-out 9 and one method doing five jobs → refactor it first

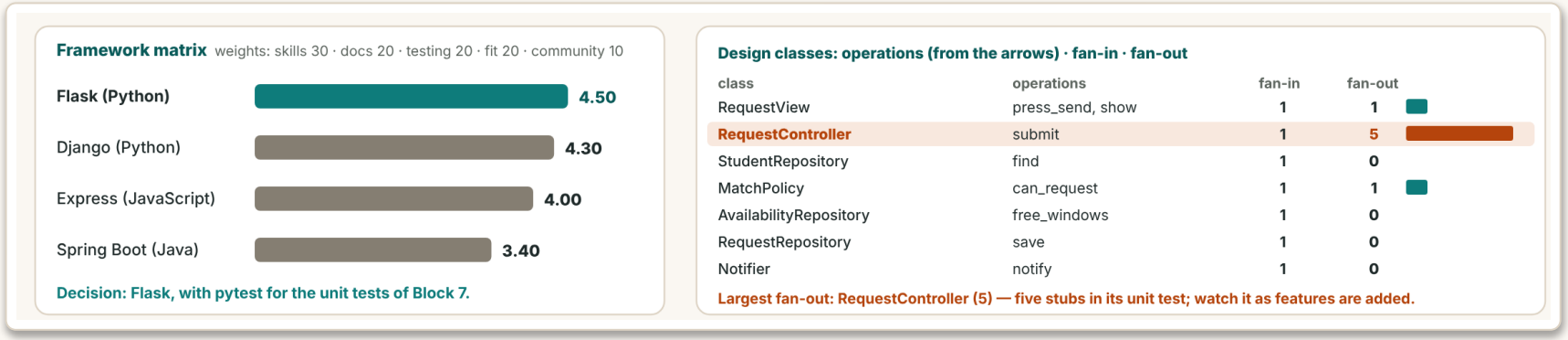


Fan-in of every class is 1. The largest fan-out — RequestController, 5 — is the first class to watch as features are added.

Example — StudyBuddy (Team Lynx, fictional)

a framework matrix, one sequence diagram, and what its arrows say

Task. Team Lynx (fictional) builds StudyBuddy: a student finds study partners by course and sends a study request; the partner confirms or declines.



Relation to the lecture. It applies Lesson 5: framework selection with a criteria matrix, the sequence diagram, the design class diagram that it fills, and the Numbers corner.

4.50

Flask, the chosen framework

8 → 7

messages → design classes with operations

5

largest fan-out: RequestController

Lesson 5 in the module — watch, see it applied, run it, check yourself

Videos — each under 15 minutes

4 checked videos; best match: “What is the difference between a Library and a Framework” (Hitesh Choudhary)

Real-life example

Team Lynx · StudyBuddy (fictional): task, relation to the lecture, deliverables that come back in later classes

Program 5 — run it step by step

demo5_studybuddy_framework_sequence.py: Start, Pause, Resume, Step, Restart; every executed step, variable and output line shown; a live visualisation

Figure 5 and the tables

Download the figure as an SVG (Scalable Vector Graphics) or PNG (Portable Network Graphics) file, and the tables as CSV (Comma-Separated Values) or the Class 6 workbook

Five flash-cards

Click to turn; shuffle; the same cards return in the Study Hub

Formative quiz — 4 + 2 questions

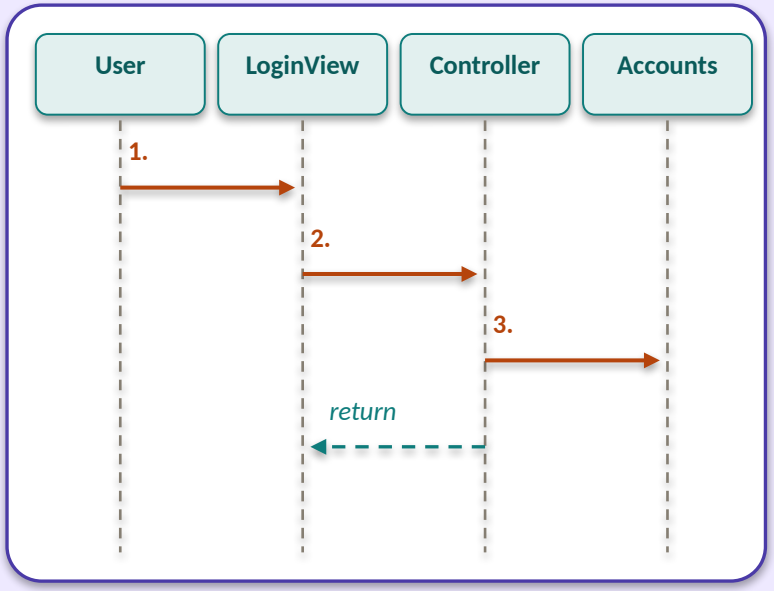
Four single-answer and two multiple-answer questions, hints, the reason for every option — formative, never graded

Active break — Try-it: A one-minute sequence diagram for “log in”, then one quiet minute

- One minute, on paper, alone: draw the sequence diagram of “log in” for your product — the user, the login view, the controller, the account repository. Number the messages; add the return arrows.
- One minute with a neighbour: is every arrow labelled with an operation? Which class has the largest fan-out?
- Last minute: one quiet minute — stand, stretch, breathe in for four counts and out for six. Nothing to do, on purpose.
- Keep the paper: the controller you drew is the first class of the code skeleton.

✋ **0:56–1:00**

four minutes · in pairs or alone



After the hour — Studio on Block 5

Time	What	Detail
1:10–1:15	Feedback patterns from the Block 5 first versions (no names)	Five minutes on two anonymised examples: one strong traceability matrix, one common gap.
1:15–2:40	Team work on Block 5, with the professor circulating	Finish it, improve it, ask questions. Today's lessons help: every domain class has one job; every requirement traces to a use case; no design class has crept into the domain model.
2:40–2:50	Optional practice, never graded: one design-pattern card; the code skeleton and the framework that Block 7 needs	For teams already done. The skeleton: folders, a README, one test that passes.
2:50–3:00	Wrap-up	The Block 5 box — final version due Sunday, December 20, 2026, 23:59; the first version for feedback was due Sunday, October 25 — and the exit ticket.

Exit ticket, one line on paper: “Which pattern, which problem?” — Block 5 final version: Sunday, December 20, 2026, 23:59.

Home Sprint — about 120 minutes, and what comes next

1. Finish Block 5 if it is not finished — the final version is due Sunday, December 20, 2026, 23:59 (course local time); the first version for feedback was due Sunday, October 25.
2. Read Pressman and Maxim, 9th edition, chapter 9 (Design Concepts), chapter 10 (Architectural Design) and chapter 14 (Pattern-Based Design); skim chapter 11 (Component-Level Design).
3. Set up the development environment and the framework skeleton — folders, a README, one test that passes — and commit it: Sprint 1 continues, and Lab 07 starts from it.
4. Flash-cards: add this week's fourteen terms to your deck and review last week's thirteen.
5. Optional, never graded: write a second pattern card for your own product.

Next: Lab 07 / Block 7 — November 3

A first test plan, the first code with at least six unit tests, a coverage figure, black-box test tables and a cyclomatic-complexity analysis — built on today's design.

First version recommended by Sunday, November 8 (block7-draft); final Sunday, December 20, 2026, 23:59; 6.5 % of the grade.

Tools in every lab are suggestions: use any suitable tool — research, compare, even build your own.

Links (1/3): course, textbook, Lessons 1 and 2

Course, legal and safety

- Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0) cyber-soho.github.io
- VirusTotal — URL (Uniform Resource Locator, a web address) scanner virustotal.com
- VirusTotal — file scanner virustotal.com
- Google Transparency Report — Safe Browsing site status transparencyreport.google.com

Textbook and body of knowledge

- Pressman and Maxim, Software Engineering: A Practitioner's Approach, 9th edition (McGraw Hill) mheducation.com
- IEEE (Institute of Electrical and Electronics Engineers) Computer Society — SWEBOK (Software Engineering Body of Knowledge) Guide, version 4.0 (2024) computer.org

Lesson 1 — design concepts

- Parnas (1972), On the criteria to be used in decomposing systems into modules doi.org
- Stevens, Myers and Constantine (1974), Structured design doi.org
- Bieman and Kang (1995), Cohesion and reuse in an object-oriented system doi.org
- Martin Fowler — Refactoring (book page) martinfowler.com

Lesson 2 — architectural design

- ISO/IEC/IEEE 42010 (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 42010):2022 — Architecture description iso.org
- ISO/IEC 25010 (International Organization for Standardization / International Electrotechnical Commission standard 25010):2023 — Product quality model iso.org
- IEEE 1016 (Institute of Electrical and Electronics Engineers standard 1016)-2009 — Software design descriptions (historical) standards.ieee.org
- James Lewis and Martin Fowler (2014) — Microservices martinfowler.com
- Martin Fowler — MonolithFirst martinfowler.com
- Michael Nygard (2011) — Documenting Architecture Decisions cognitect.com
- Architecture decision records — community site adr.github.io
- Government of Canada — Enterprise Architecture Framework canada.ca

Official sites only, checked on October 25, 2026. Check any address with VirusTotal before you open it — Legal Notice section 14 and Annex C.

Links (2/3): Lessons 3, 4 and 5

Lesson 3 — design principles

- Robert C. Martin (2014) — The Single Responsibility Principle blog.cleancoder.com
- Liskov and Wing (1994), A behavioral notion of subtyping doi.org

Lesson 4 — design patterns

- Gamma, Helm, Johnson and Vlissides (1994), Design Patterns (Inform IT, Addison-Wesley) informit.com

Lesson 5 — frameworks, UML (Unified Modeling Language) and interfaces

- Flask flask.palletsprojects.com
- Django djangoproject.com
- Express expressjs.com
- Spring Boot spring.io
- OMG (Object Management Group) — UML (Unified Modeling Language) 2.5.1 specification omg.org
- Mermaid — sequence diagram syntax mermaid.js.org
- Mermaid — class diagram syntax mermaid.js.org
- PlantUML plantuml.com
- Henry and Kafura (1981), Software structure metrics based on information flow doi.org
- W3C (World Wide Web Consortium) — WCAG (Web Content Accessibility Guidelines) 2.2 w3.org
- Nielsen Norman Group — 10 usability heuristics for user interface design nngroup.com
- Python python.org

Official sites only, checked on October 25, 2026. Check any address with VirusTotal before you open it — Legal Notice section 14 and Annex C.

Links (3/3): videos — each under 15 minutes

Lesson 1

- Coupling and Cohesion Udacity · ≈ 3 min
- Abstraction Can Make Your Code Worse CodeAesthetic · ≈ 5 min
- Why You Shouldn't Nest Your Code CodeAesthetic · 8:29 min

Lesson 2

- Top 5 Most Used Architecture Patterns ByteByteGo · ≈ 6 min
- MVC Explained in 4 Minutes Web Dev Simplified · ≈ 4 min
- What are Microservices? IBM (International Business Machines) Technology · 6:38 min
- Lesson 55 - Architecture Decision Records Software Architecture Monday (Mark Richards) · 10:28 min

Lesson 3

- SOLID Principles: Do You Really Understand Them? Alex Hyett · ≈ 7 min
- 7 Principles Every Developer Must Know Hayk Simonyan · 5:34 min

Lesson 3

- Dependency Injection, The Best Pattern CodeAesthetic · 13:15 min

Lesson 4

- 10 Design Patterns Explained in 10 Minutes Fireship · 11:04 min
- 8 Design Patterns EVERY Developer Should Know NeetCode · ≈ 9 min
- The Observer Pattern Explained and Implemented in Java | B... Geekific · 6:48 min

Lesson 5

- What is the difference between a Library and a Framework Hitesh Choudhary · 6:04 min
- How to Make a UML Sequence Diagram Lucid Software · ≈ 9 min
- UML Class Diagram Tutorial Lucid Software · ≈ 10 min
- Jakob Nielsen's 10 Usability Heuristics IxDF (Interaction Design Foundation) · 1:01 min

YouTube, checked October 25, 2026. MVC = Model–View–Controller · SOLID = five design principles · UML = Unified Modeling Language · IBM = International Business Machines.

Legal notice, credits and the rules that apply

Use of this material is governed by the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0). CS 330 is not a cybersecurity course: nothing here monitors, scans or tests a system, so section 7 of the Notice does not arise.

- **§ 4** Licence and copyright — personal study, no re-upload
- **§ 4.6** Third-party content: frameworks, videos, books, standards
- **§ 5** No affiliation with any vendor or standards body
- **§ 6.6** Open-source tools keep their own licences
- **§ 6.7** Fictional or synthetic data only
- **§ 10** You run the demo programs on your own equipment
- **§ 14** Check every link and file before you trust it
- **§ 17** No recording of sessions without written permission
- **§ 18** Academic integrity — your own work

Credits

Pressman and Maxim, *Software Engineering: A Practitioner's Approach*, 9th edition — chapters 9–12 and 14.

Gamma, Helm, Johnson and Vlissides, *Design Patterns* (1994).

Parnas (1972); Stevens, Myers and Constantine (1974); Bieman and Kang (1995); Liskov and Wing (1994); Henry and Kafura (1981).

Teams, products, people and numbers in the examples are fictional.